



GaiaCom

User, Operator and Developer Manual

Sovereign Beta v2 · System & Architecture · September 2026

Post-Quantum Communication Infrastructure · Zero Server Trust · Federated by Design

GaiaCom - User, Operator and Developer Manual

Documentation status: Sovereign Beta v2 / Repository snapshot September 2026

Product: GaiaCom

Document type: User Manual · Node Operator Manual · Deployment Guide · Developer Reference

Basis: Source code and repository documentation of the provided `GaiaCom-main` snapshot together with the separately supplied detailed architecture map.

Important: This manual describes the repository state actually provided. Wherever source code, README, older documents and the architecture map differ, the discrepancy is explicitly identified. In this English edition, the separately supplied architecture map is included in **Appendix A as a complete English translation.**

Table of Contents

1. About this Manual
2. GaiaCom in One Sentence
3. Product Status and Release Boundaries
4. Source of Truth and Known Documentation Drift
5. Security Model and Trust Boundaries
6. Core Terms

Part I - User Manual

1. First Start, Registration and Onboarding
2. Login, Local Lock and Recovery
3. Pairing a New Device
4. Navigation and Dashboard
5. Identity, GaiaID, Contacts and Trust Passport
6. Quantum Chat / Direct Messaging
7. GaiaMail
8. SMTP Legacy Bridge
9. Groups, Rooms and Subchannels
10. Public Channels
11. GSN - GaiaSocialNetwork
12. GaiaDrive
13. GaiaDrop
14. Network Health
15. Security Center / GaiaShield
16. Abuse Center, Governance and Gaia's Eyes
17. Profile and Personal Settings
18. Keyboard Shortcuts

Part II - Node Operator and Deployment Manual

1. Operating Model
2. Prerequisites
3. Local Development Environment
4. Production Build
5. Preparing the Linux Server
6. Production Configuration with `gaiacom setup`
7. `gaiacom doctor` and `gaiacom version`
8. Governance Bootstrap
9. Systemd Hardening

10. NGINX, TLS and Reverse Proxy
11. Firewall and Network Exposure
12. Node Registry and Controlled Federation
13. Storage: Local Object Store and S3/MinIO
14. Configuring SMTP for Production
15. Health Checks, Metrics and Logs
16. Backups and Recovery
17. Updates and Rollback
18. Production Checklist

Part III - Developer Manual

1. Repository Structure
2. Frontend Architecture
3. Backend Architecture
4. Persistence and SQLite
5. Sovereign Crypto Core, WASM and Desktop
6. Android Platform
7. API Overview
8. Security and Quality Gates
9. Developing and Extending Securely

Part IV - Operations Support

1. Troubleshooting
2. Known Limitations of Sovereign Beta v2
3. Glossary
4. Documentation Maintenance

Appendix

- A. Complete Architecture Map
-

1. About this Manual

GaiaCom is not a single web application. The provided state includes a React/Vite interface, a Go backend, SQLite persistence, a Rust-based Sovereign Crypto layer, WebAssembly, a Tauri desktop host, a modular Android platform, NGINX/systemd deployment files, federation logic, governance and security subsystems, and adversarial test suites.

This manual is therefore written for four groups of readers:

- **End users** who use GaiaCom.
- **Administrators and node operators** who install, secure and operate a GaiaCom node.
- **Developers** who need to understand or extend the repository.
- **Security reviewers** who need to assess security boundaries, test gates and known limitations.

The architecture map in Appendix A serves as the in-depth technical reference. The main body of this manual focuses on **usage, setup, operation and practical workflows**.

2. GaiaCom in One Sentence

GaiaCom is a **federated communications platform with client-side end-to-end encryption and post-quantum-oriented hybrid cryptography** that combines direct messaging, mail, group rooms, public channels, social features, encrypted file storage, anonymous or pseudonymous text submissions, governance and node federation in one platform.

The central design principle is **Zero Server Trust**: a correctly operated GaiaCom server should not need confidential user content in plaintext in order to provide transport, delivery, federation and storage.

3. Product Status and Release Boundaries

The provided snapshot must be treated as **Sovereign Beta v2**. Not every UI element or experimental component visible in the source code is automatically an approved production feature.

3.1 Active or intended in the beta state

Area	Status in the provided state
Native E2EE Direct Messages	Active
Group rooms and subchannels	Active
GaiaMail	Active
SMTP Bridge	Active, but an explicit security downgrade
GaiaDrive	Active
GaiaDrop text submissions	Active
GaiaProof	Active
TrustMesh	Active, local/policy-based
Trust Passport / Gaia Passport	Active
Key-change warnings	Active
Public Channels	Active
GSN Social Layer	Active
Controlled Federation	Active, known/approved nodes
Node Registry MVP	Active
Governance / Reviewer	Active
GaiaShield / Security Center	Active
Sovereign Accelerated	Active
Sovereign Top Secret	Active only when required capabilities are satisfied
Browser HQC provider	Active through isolated worker/WASM
Tauri Native Crypto Bridge	Active
S3/MinIO Object Store Adapter	Optionally active
Android modular platform base	Technical beta

3.2 Intentionally not yet to be treated as fully released

The release documentation excludes, among other things, the following areas from full release status:

- File, media and image attachments in chat until the attachment audit has been completed.
- Open public federation without controlled admission.
- Completely open node registration.

- Globally synchronized abuse consensus.
- A complete Mobile Sovereign v1 messaging path.
- Triple Ratchet or ratchet-based forward/post-compromise secrecy.
- Identity-signed per-device Sovereign prekeys as a fully completed path.
- Attachment-based GaiaDrop.
- A fully automated update pipeline.

Practical rule: If the UI already exposes a button for an experimental feature but the release matrix still marks that feature as disabled or unaudited, the **release boundary** takes precedence.

4. Source of Truth and Known Documentation Drift

While creating this manual, the README, source code, deployment files and separately supplied architecture map were checked against one another. Several differences are visible that matter for installation and security.

4.1 Password hashing: Argon2id

GaiaCom uses **Argon2id** as the mandatory server-side password-hashing standard. Registration, login, password change and account deletion must all consistently use the same central Argon2id path.

For new and migrated accounts, Argon2id is the target and release standard. Any remaining legacy hashes are to be migrated in a controlled manner during the ongoing transition; new password hashes are created exclusively with Argon2id. The architecture map in Appendix A therefore matches the intended cryptographic target state.

4.2 Vite configuration file

The architecture map sometimes refers to `vite.config.js`. In the provided repository, the file is:

```
Frontend/frontend/vite.config.mjs
```

4.3 License: GNU AGPLv3

GaiaCom is published consistently under the **GNU Affero General Public License v3 (AGPLv3 / AGPL-3.0-only)**. `LICENSE`, README, package metadata and source-code notices should therefore consistently identify AGPLv3.

4.4 Do not assume `OPEN_REGISTRATION=false` is a secure user-registration gate

`gaiacom setup` writes `OPEN_REGISTRATION=false` in the production profile if no value exists. In the reviewed authentication registration path, however, this variable is not clearly visible as an authoritative server-side gatekeeper for `RegisterUser`.

Operational recommendation: Do not rely on this variable alone when public user registration must be disabled. Implement and test the required gate explicitly in the authentication path.

4.5 Local frontend/backend ports

The frontend script starts Vite by default with:

```
vite --host 127.0.0.1
```

Vite typically uses port 5173. The backend development CORS path in the snapshot, by contrast, is tailored to local origins such as `http://localhost:3000` and `http://localhost:1420`.

For reproducible development, this manual therefore explicitly uses port **3000**.

4.6 Desktop API is pinned to `beta.gaiacom.de` in the source

For desktop mode, the frontend API client contains:

```
https://beta.gaiacom.de
```

The Tauri CSP is also aligned with that target. Anyone shipping a desktop build for a custom node must **deliberately adjust the API origin and CSP and rebuild the client**.

4.7 Production provisioning: CLI before UI secret generator

Some older operator documents describe generating node secrets through the Security Center / Nodesystem area. The current production path in the repository instead uses `gaiacom setup`. It creates secrets atomically, writes the environment with restrictive permissions and can then be checked with `gaiacom doctor`.

This manual treats `gaiacom setup` as the canonical production path.

4.8 Backend build entry point

The actual entry point for the executable server is:

```
Backend/cmd/gaiacom
```

The production build should therefore use `./cmd/gaiacom` rather than a generic build of the backend package root.

5. Security Model and Trust Boundaries

5.1 What GaiaCom is designed to protect

The system is designed to support the following protection goals:

- Confidentiality of native messages and files through client-side encryption.
- Authenticity and integrity of cryptographic envelopes.
- Resistance to downgrade and key-substitution attacks.
- Signed and replay-resistant server-to-server communication.
- Minimization of server-side plaintext.
- Limitation of operator and reviewer privileges.
- Traceable security events through GaiaShield and audit chains.
- Cryptographically protected trust/disclosure workflows.

5.2 What GaiaCom explicitly does not solve automatically

Zero Server Trust does not mean "Zero Risk."

A compromised or unlocked endpoint can expose plaintext, session material or user input. A compromised operating system can enable keylogging, screen capture or memory access. The Web/PWA variant also relies on the JavaScript application currently delivered by the node not being maliciously replaced.

For high assurance, a **signed, reproducibly built native client** is conceptually stronger than a web client that is freshly loaded from the node on every visit.

GaiaCom is also **not an anonymity network**. Federation, delivery, timing, account and network metadata may remain sensitive even when message content is encrypted.

5.3 Native communication versus SMTP

Native GaiaCom communication remains within GaiaCom cryptography and the GaiaCom federation model.

With SMTP, the message leaves this trust boundary. The bridge must therefore be treated in both UI and operations as a **deliberate downgrade**. Once handed to traditional mail infrastructure, that infrastructure's transport, provider and metadata risks apply.

6. Core Terms

GaiaID

A federated user address in the form:

```
@username:node.example.org
```

Among other things, the node part determines where federated communication must be routed.

Sovereign Accelerated

The higher-performance Sovereign mode. In the current design it combines a hybrid KEM derivation with a two-stage authenticated encryption cascade.

Sovereign Top Secret

A higher cryptographic tier with an additional cipher cascade and additional signature/capability requirements. It may only be enabled when both communication endpoints and the message path demonstrably support the required capabilities.

GaiaProof

A proof format that can cryptographically substantiate relevant communication or abuse events without introducing a universal administrative decryption key.

TrustMesh

A local, federation-related reputation and friction system. It is not a global "truth score."

Gaia Passport / Trust Passport

A presentation of identity, Human Proof and trust information associated with a GaiaID.

Gaia's Eyes

A controlled disclosure workflow for evidence voluntarily or policy-conditionally supplied in governance/reviewer processes.

GaiaShield

A security subsystem for anomalies, audit events, protective middleware, redaction and operator visibility.

Part I - User Manual

7. First Start, Registration and Onboarding

7.1 Start screen

The authentication area provides several entry paths:

- Login
- Registration
- Recovery
- Device pairing
- GaiaDrop for external text submissions without a normal account login

7.2 Registration

For a new account, the UI requires at least a username, a password and the required legal and/or beta confirmations.

The backend state enforces server-side plausibility limits for usernames and passwords. In the current state, passwords have a minimum length of **12 characters**.

During registration, cryptographic identity material is generated **client-side**. This includes recovery/mnemonic material, which the server is not intended to possess as a plaintext backup.

7.3 SetupWizard - the five essential phases

Phase 1 - Terms and beta notices

The user confirms, among other things:

- The terms have been read.
- The beta status is understood.
- There is no guarantee of permanent availability.
- SMTP lies outside the full native security model.
- Responsibility for recovery/credential safekeeping.
- Lawful use.

Phase 2 - Securing recovery

The recovery phrase is displayed locally. The user must confirm that it has been backed up.

Recommendation: Never store recovery material in chat, email, cloud notes or screenshots unless that storage location has deliberately been chosen as a secure recovery location.

Phase 3 - Identity and GaiaID

The user creates or registers an identity on a node. This produces the federated GaiaID.

Example:

Depending on the UI state, available nodes, a self-operated node or fallback nodes may be offered.

Phase 4 - Security introduction

The onboarding explains the core layers:

- Identity
- Message encryption
- Network/federation
- Transparency and trust

Phase 5 - Getting started

Typical quick destinations are:

- Inbox
- Contacts
- Channels
- Security Center

7.4 First-run onboarding

In addition to SetupWizard, there is a first-run onboarding flow for profile, avatar, security notices, backup confirmation and the user's first transition into the application.

8. Login, Local Lock and Recovery

8.1 Login

A normal login authenticates the user against the backend and initializes local client state. Access and refresh tokens are handled separately from the actual recovery material.

8.2 Local session lock

GaiaCom can locally lock an already authenticated interface after inactivity. Depending on the active configuration, unlocking can be performed using:

- Password
- PIN
- WebAuthn PRF / hardware-bound derivation

The local lock is not the same as a full logout.

8.3 Logout

Logout terminates the current server-side session or the associated token path and clears the local authentication state.

8.4 Importing a recovery file

The recovery area can import encrypted GaiaCom recovery files, for example in `.json` or `.gaiacom-recovery.json` format.

Typical flow:

1. Select the recovery file.
2. Enter the recovery-file password.
3. Set a new local unlock password.
4. Decrypt the recovery data.
5. Restore mnemonic/identity material locally.
6. Reinitialize account/identity state.

Important: A recovery file is only as secure as its password and storage location.

9. Pairing a New Device

GaiaCom separates **device pairing** from an ordinary browser/login session.

9.1 On the new device

In the pairing area:

1. Enter account details.
2. Assign a device name or label.
3. Request pairing.
4. GaiaCom generates a one-time pairing code or URI in the form:

```
gaiacom://pairing/<pairing-id>/<one-time-secret>
```

This can be transferred as a code/QR to the already trusted device.

9.2 On the already trusted device

In the profile under **Devices**:

1. Paste or scan the pairing code.
2. Review the pairing.
3. Select the active identity.
4. Recovery/Sovereign material is cryptographically wrapped for the new device.
5. Sign the approval.
6. Confirm the pairing.

9.3 Completing pairing on the new device

The new device checks whether approval is available, consumes the pairing and stores the released local key material in protected form.

9.4 Managing devices later

Depending on state, the profile allows users to:

- revoke active sessions,
- revoke registered crypto devices,
- rotate current device keys,
- approve new pairings.

If a device is lost, both **sessions and device keys** should be reviewed and revoked where necessary.

10. Navigation and Dashboard

The main navigation is divided into three large areas.

10.1 Workspace

- Dashboard
- GaiaMail
- SMTP Mail
- Quantum Chat
- Group Chats

GaiaMail and SMTP provide mailbox folders such as:

- Inbox
- Drafts
- Sent
- Starred
- Important
- Snoozed
- Archive
- Spam
- Trash

10.2 Network

- Public Channels
- GSN Social
- Network Health
- Security Center
- Abuse Center

10.3 Personal

- Address Book / Contacts
- My Profile
- GaiaDrive
- GaiaDrop

At the bottom of the navigation area, depending on the client, there are controls for theme, language, lock, logout, Quantum Shield status and version information.

10.4 Dashboard

The dashboard is the command center. Among other things, it shows:

- Session/security status
 - Network/uplink status
 - Active identity
 - Cryptographic status indicators
 - Connected rooms
 - Current or new mail/chat activity
 - Quick actions
 - PWA installation notice, where applicable
-

11. Identity, GaiaID, Contacts and Trust Passport

11.1 Adding contacts

Contacts can be searched for or added through a GaiaID.

Example:

```
@alice:node.example.org
```

The GaiaID is formally validated before being used as a federated address.

11.2 Contact profile

The contact profile brings together:

- GaiaID
- Profile information
- Trust/passport data
- Human Proof status
- Key/fingerprint information
- Key history

11.3 Verifying fingerprints

For particularly sensitive contacts, the displayed fingerprint should be compared **through a second, already trusted channel**.

11.4 Key Change Warning

GaiaCom maintains a local, append-only-oriented key history. If an expected recipient key changes, the client can issue a warning.

A warning does not automatically mean an attack. Possible legitimate causes include:

- a new device,
- key rotation,
- account recovery,
- reinstallation.

For important contacts, the change should nevertheless be verified.

11.5 Human Proof

The Human Proof workflow creates or renews a cryptographic proof of human interaction. It is part of the Passport/Trust system and is not a universal legal identity verification mechanism.

12. Quantum Chat / Direct Messaging

12.1 Opening a chat

Open Quantum Chat and select a contact. The header may show identity, presence, trust/role information and E2EE/Quantum status.

12.2 Sending a message

Native chat messages are transformed client-side into an encrypted envelope before upload. The backend routes or stores the envelope data without needing to decrypt the message content for the normal delivery path.

12.3 Accelerated and Top Secret

Accelerated

Suitable as the regular Sovereign mode.

Top Secret

A stronger cascade with stricter capability requirements.

The client must **fail closed**: if required primitives, recipient keys or capability proofs are missing, it must not silently fall back to a weaker variant.

12.4 Presence and typing state

The chat supports heartbeat/presence and typing indicators. These are metadata functions; they must not be confused with content encryption.

12.5 Message actions

Depending on the message and permissions:

- Reply
- Edit
- Delete
- Reactions
- Save locally
- Pin locally
- Read state

12.6 Searching a chat

The interface includes search functions for the current conversation. Where search is performed locally on decrypted data, it should not be misunderstood as server-side plaintext search.

12.7 Blocking or reporting a contact

Contacts can be blocked in chat or reported through the abuse workflow.

12.8 GaiaProof / Export

GaiaProof/export functions may be available for relevant communication histories. Encrypted exports should be protected with a separate strong export password.

12.9 Attachments and voice notes

The source code contains UI and development components for files, GaiaDrive sharing and voice notes.

Release boundary: Under the Sovereign Beta v2 contract, chat file/media attachments are not yet to be treated as fully released. For production documentation, they remain experimental until the planned audit has been completed and the release matrix has been updated.

13. GaiaMail

GaiaMail provides a mailbox-oriented communication model inside the GaiaCom application.

13.1 Composing a new mail

Open the Composer and use native GaiaCom mode.

Typical fields:

- Recipient
- Subject
- Message body
- Formatting actions such as bold, italic, underline, heading and code

For native recipients, the interface uses GaiaID/contact information.

13.2 Drafts

The Composer supports draft management and automatic or periodic saving of draft state.

13.3 Scheduled sending

The backend and Composer contain a scheduler path for delayed sending.

For scheduled messages, the node must be operational at the relevant time and able to process the scheduled delivery.

13.4 Reading a message

Depending on the message type, the Reader provides features including:

- Reply
- Reply all
- Forward
- Archive
- Delete
- Spam
- Snooze
- Labels
- GaiaProof/disclosure functions
- Sender/recipient profile

13.5 Warning banners

The Reader can display warnings for:

- Spam/quarantine
- SMTP legacy/downgrade
- Changed contact keys

13.6 Filter rules

Mailbox rules can apply actions based on criteria such as sender or subject, for example:

- Important
- Star
- Read
- Label

13.7 Signature and mailbox settings

The profile provides mailbox settings such as signature, partial locale settings and keyboard-oriented operating options.

14. SMTP Legacy Bridge

The SMTP Bridge enables interoperability with traditional email.

14.1 Security meaning

SMTP is **not a native GaiaCom E2EE path**. As soon as a message is handed to the legacy gateway, GaiaCom cannot enforce its security guarantees outside its own infrastructure.

The UI must visibly mark this transition as a downgrade.

14.2 Composing an SMTP message

Select SMTP/Legacy in the Composer. External email addresses are then permitted.

Before sending, the user should consciously acknowledge the downgrade warning.

14.3 Operator obligations

A production SMTP setup additionally requires:

- authenticated SMTP relay
- mandatory TLS
- a clean sender domain
- SPF
- DKIM
- DMARC with a meaningful enforcement policy
- protection against header/CRLF injection
- monitoring for abuse/rate limits

15. Groups, Rooms and Subchannels

15.1 Creating a room

A new room can be created through **Group Chats**.

Possible properties:

- Name
- Description
- Avatar
- Private Room
- Crisis Room
- Top Secret

15.2 Private room

A private room should not be discoverable like a normal public room. Joining can take place through an invitation or join request.

15.3 Subchannels

Channels can be created within a room to separate topics.

15.4 Slow Mode

The group area supports Slow Mode intervals including:

0 / 5 / 10 / 30 / 60 / 300 seconds

15.5 Members and roles

Depending on permissions:

- Member
- Admin
- Owner

Room operators can search members, change roles, remove members and transfer ownership.

15.6 Join requests

For private rooms, join requests can be accepted or rejected.

15.7 Invitation links

Invitations can be limited by time or number of uses, for example:

- 1 hour
- 1 day
- 7 days
- unlimited

and:

- 1 use
- 5 uses
- 10 uses
- unlimited

15.8 Moderation logs

Room-related administration/moderation actions are recorded for the intended auditability.

16. Public Channels

Public Channels are public broadcast/discussion areas.

16.1 Finding and subscribing to channels

Users can discover, search, subscribe to and unsubscribe from channels.

16.2 Posts

Depending on permissions:

- Create post
- Edit post
- Delete post
- Pin post
- Add reaction

16.3 Comments

Channels can allow or disable comments. Moderators can manage comments.

16.4 Moderation and reports

Channels or content can be reported. Categories in the system include, for example:

- Spam
- Phishing
- Malware
- Harassment
- Illegal content
- Threat
- Other

A report is a governance signal, not automatic proof.

17. GSN - GaiaSocialNetwork

GSN is GaiaCom's decentralized social-layer component.

17.1 Feeds

Depending on the view:

- Node/general feed
- Following feed

17.2 Posts

Users can:

- create posts,
- delete their own posts,
- react,
- comment.

17.3 Profiles and follows

Depending on settings, GSN profiles contain fields such as:

- Display name
- Bio
- Website
- Avatar

Other profiles can be followed or unfollowed.

17.4 Reporting

Profiles and posts can be reported into the abuse/governance path.

17.5 Connection to other GaiaCom areas

The UI links social profiles with chat, rooms and public channels so users can move between public and private communication.

18. GaiaDrive

GaiaDrive is the current encrypted personal storage area. The older `VaultPane/useVault` path should be treated as legacy code.

18.1 Unlocking Drive

The user unlocks GaiaDrive locally with the Drive password. This password is used for local key derivation and should not be treated as a weak convenience password.

18.2 Notes

GaiaDrive can manage encrypted local notes.

Possible actions:

- Create note
- Assign category
- Display decrypted
- Copy
- Delete

18.3 Local files

Files are managed locally through browser/client storage mechanisms such as OPFS and stored in encrypted form.

18.4 Cloud synchronization

For supported files, the client can upload client-side-encrypted chunks to the node. The server sees encrypted chunks and metadata, not decrypted file content.

The current UI applies additional size/TTL boundaries to certain synchronization functions. These limits should be synchronized with server configuration and README before release.

18.5 Cross-device sync

The complete key-distribution and recovery ceremony for seamless cross-device GaiaDrive synchronization remains a known beta limitation.

19. GaiaDrop

GaiaDrop is designed for external text submissions to a GaiaID.

19.1 Submission without a normal account login

From the authentication/public entry point:

1. Select GaiaDrop.
2. Enter the recipient GaiaID.
3. Write the text.
4. Submit it.

The current release path is **text-oriented**. Attachment-based GaiaDrop is not yet to be treated as a released feature.

19.2 GaiaDrop inbox

The recipient opens the GaiaDrop area and loads their submissions.

Details may include:

- Timestamp
- Drop ID
- Payload hash
- Sender label or anonymized state, where available
- The text after client-side decryption

19.3 Security boundary of "anonymous"

GaiaDrop is intended to enable low-friction submissions that are not necessarily tied to a normal account. This must not be marketed as a mathematical network-anonymity guarantee. Network and operational metadata lie outside such a guarantee.

20. Network Health

Network Health presents aggregated network/node state.

Typical data includes:

- Accounts
- Identities
- Nodes
- Rooms
- Messages within a time window
- GaiaDrop activity
- Federation activity
- Uptime
- Protocol version
- Crypto/capability status
- Federation status
- SMTP Bridge status

The profile contains a privacy setting for anonymous or aggregated telemetry. Personal communication graphs or GaiaIDs should not be treated as normal public health metrics.

21. Security Center / GaiaShield

The Security Center is the user and operator interface for GaiaShield and related security functions.

21.1 Protection Status

Depending on permissions, it can show:

- Account protection
- Open warnings
- Failed login attempts
- Message/replay anomalies
- Security checks

Warnings can be acknowledged after review.

21.2 Security Log

Security events can be filtered by severity/category and exported for analysis, for example as JSON or CSV.

21.3 Gaia Passport

The Passport area shows Human Proof and trust information, among other things. A Human Proof can be renewed.

21.4 Node Security

Operators receive additional node-related events and statistics.

By design, GaiaShield captures areas including:

- Authentication attacks
- Malformed requests
- BOLA/BFLA attempts
- Storage ACL violations
- Federation anomalies
- SMTP abuse
- Governance abuse
- GSN spam
- Upload abuse

Sensitive fields are redacted or hashed before persistence or display where the security subsystem specifies it.

21.5 Nodesystem

The Nodesystem area presents registry/node information and diagnostic actions.

Production rule: Do not provision node secrets primarily through the browser UI. For a production node, use `gaiacom setup` and `gaiacom doctor`.

22. Abuse Center, Governance and Gaia's Eyes

22.1 Roles

Depending on the bootstrap/credential model, the governance layer distinguishes roles such as:

- User
- Reviewer
- Senior Reviewer
- Node Operator

Role permissions must be checked server-side.

22.2 Own reports

Users can see submitted cases and their status. Depending on the workflow, an appeal can be filed.

22.3 Reviewer Queue

Reviewers receive policy-based cases and, after server-side authorization checks, can sign assessments or proposed actions.

Possible measures may include:

- reject
- quarantine
- suspend

The exact threshold logic depends on policy/governance configuration.

22.4 Gaia's Eyes

Gaia's Eyes is not a "Godmode." The workflow is intended to enable controlled, limited disclosure when the affected client or the designated cryptographic disclosure logic supplies the required material.

22.5 Node-operator actions

Operators can apply node/abuse actions within their governance scope and publish transparency information.

23. Profile and Personal Settings

23.1 Editing the profile

Possible fields include:

- Display name
- Optional real name
- Website
- Bio/status
- Avatar

Uploads and profile content must remain sanitized or validated on both client and server sides.

23.2 Changing the password

Required:

- Current password
- New password
- Confirmation

The server password is not the same as the recovery mnemonic or a local GaiaDrive/export password.

23.3 Devices

See Chapter 9. Sessions and crypto devices can be managed from the profile.

23.4 Keys

After reauthentication, depending on the UI, the client can display local private keys and the mnemonic.

This area is particularly sensitive. Screen sharing, browser extensions, malware or physical access to an unlocked device can defeat the intended protection.

23.5 Exporting recovery data

Recovery files are exported in encrypted form. Use a separate strong password and store the file offline or separately from the password.

23.6 Crypto Lock

Configurable elements may include:

- Crypto-session duration
- Inactivity auto-lock

- PIN unlock
- WebAuthn PRF

23.7 Privacy

The user can configure participation in anonymized/aggregated network statistics.

23.8 Accessibility

The client provides, among other things:

- Font Scale: 90%, 100%, 112%, 125%
- High Contrast
- Reduce Motion

23.9 Notifications

Depending on the client:

- Notifications on/off
- Content preview
- Quiet Hours
- Start/end time

23.10 Language

The global i18n layer of the provided frontend contains 18 language options:

Deutsch, English, Русский, Español, Français, فارسی, 日本語, Português, العربية, 简体中文, हिन्दी, Türkçe, Italiano, Polski, Українська, 한국어, Indonesia and Shqip.

Some subdialogs still expose smaller locale lists than the global client.

23.11 Theme

Light/Dark mode can be switched through the navigation.

23.12 Deleting the account

The Danger area requires explicit confirmation and the current password.

Deletion must be treated as irreversible. Required recovery/export data should be secured beforehand.

24. Keyboard Shortcuts

The client contains Gmail-style navigation/work shortcuts. They are not triggered while typing in suitable input fields.

Shortcut	Action
g then i	Inbox
g then s	Sent
g then c	Chat
g then g	Groups
g then p	Profile
c	Compose
r	Reply
f	Forward
e	Archive
Delete, Backspace or #	Trash
s	Toggle Star

Part II - Node Operator and Deployment Manual

25. Operating Model

In the recommended Linux path, a production GaiaCom node consists of:



Port `8080` should **not be exposed directly to the Internet** in the standard production model.

26. Prerequisites

26.1 Build system

Depending on the component being built:

- Go toolchain matching the repository (`go 1.25.0` , toolchain note `go1.26.5`)
- Node.js matching `.node-version`
- npm
- Rust/Cargo
- `wasm-pack` for WASM
- Tauri toolchain for desktop
- Python for hygiene/security scripts
- Android Studio/Gradle/JDK for Android

26.2 Production server

Recommended for the provided deployment path:

- Linux AMD64
 - NGINX
 - systemd
 - TLS certificate
 - dedicated `gaiacom` system user
 - local SQLite on persistent storage
 - separate persistent GaiaCom storage path
 - DNS/FQDN
-

27. Local Development Environment

27.1 Repository

Unpack or clone the repository and work from its root.

27.2 Backend environment

The backend does not automatically load a `.env` file like a typical `godotenv` application. Variables must be exported into the process environment or loaded by the shell/service.

A minimal **local** example:

```
cd Backend

export GAIACOM_DEV_MODE=true
export SERVER_BIND_ADDRESS=127.0.0.1
export SERVER_PORT=8080
export DB_DRIVER=sqlite
export DB_PATH="$PWD/gaiacom-dev.db"
export GAIACOM_COOKIE_SECURE=false

go mod verify
go test ./...
go run ./cmd/gaiacom
```

In development mode, production secrets must not be reused as real deployment secrets.

27.3 Frontend

For the backend's existing development-origin configuration, port `3000` is the more robust path:

```
cd Frontend/frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund

VITE_API_URL=http://localhost:8080 \
npx vite --host localhost --port 3000
```

PowerShell equivalent:

```
cd Frontend\frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund
$env:VITE_API_URL='http://localhost:8080'
npx vite --host localhost --port 3000
```

27.4 Why not simply `npm start`?

The existing script is:

```
vite --host 127.0.0.1
```

Without an explicit port, it may use Vite's default port while the backend development CORS state allows different origins. A reproducible developer guide should therefore specify host and port deliberately.

28. Production Build

28.1 Frontend

Bash:

```
cd Frontend/frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund
npm test
npm run test:hqc-browser
VITE_API_URL=https://node.example.org npm run build
```

PowerShell:

```
cd Frontend\frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund
$env:VITE_API_URL='https://node.example.org'
npm test
npm run build
npm run test:hqc-browser
```

Output:

```
Frontend/frontend/dist/
```

28.2 Backend Linux AMD64

```
cd Backend
go mod verify
go test ./...

VERSION=2.0.0
COMMIT="$(git rev-parse HEAD)"
BUILT_AT="$(date -u +%Y-%m-%dT%H:%M:%SZ)"

CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \
go build -trimpath -buildvcs=false \
-ldflags="-s -w \
-X gaiacom/backend/buildinfo.Version=${VERSION} \
-X gaiacom/backend/buildinfo.Commit=${COMMIT} \
-X gaiacom/backend/buildinfo.BuiltAt=${BUILT_AT}" \
-o gaiacom-backend-linux-amd64 ./cmd/gaiacom
```

Release metadata is security/operations-relevant in the production path. A production server should not be started with an invalid development release marker.

29. Preparing the Linux Server

Example for Debian/Ubuntu:

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y nginx certbot python3-certbot-nginx

sudo useradd -r -s /bin/false -d /opt/gaiacom gaiacom

sudo install -d -o root -g root -m 0755 /opt/gaiacom
sudo install -d -o root -g root -m 0755 /var/www/gaiacom
sudo install -d -o root -g root -m 0700 /etc/gaiacom
sudo install -d -o gaiacom -g gaiacom -m 0700 /var/lib/gaiacom
sudo install -d -o gaiacom -g gaiacom -m 0700 /opt/gaiacom/storage
```

Then transfer the backend binary and frontend in a controlled manner. The binary and frontend should not be writable by the unprivileged runtime user unless this is strictly necessary.

30. Production Configuration with `gaiacom setup`

30.1 Canonical setup

```
sudo /opt/gaiacom/gaiacom-backend setup \
  --config /etc/gaiacom/gaiacom.env \
  --server-name node.example.org \
  --db-path /var/lib/gaiacom/gaiacom.db \
  --port 8080
```

Afterwards:

```
sudo chmod 600 /etc/gaiacom/gaiacom.env
sudo chown root:root /etc/gaiacom/gaiacom.env
```

30.2 What `setup` sets or creates

The current provisioning path sets, among other things:

```
GAIACOM_DEV_MODE=false
GAIACOM_SERVER_NAME=<FQDN>
SERVER_BIND_ADDRESS=127.0.0.1
GAIACOM_ALLOW_PUBLIC_BIND=false
DB_DRIVER=sqlite
GAIACOM_COOKIE_SECURE=true
```

and creates or preserves valid secrets for:

- JWT
- GaiaShield
- Metrics
- TrustMesh Epoch
- Ed25519 Server Identity

The configuration file is created atomically with restrictive permissions.

30.3 Idempotence

`setup` is designed so that valid existing secrets are not replaced on every run. A server-name mismatch or invalid existing secrets should not be silently overwritten.

Therefore: Do not generate new secrets before every start "just to be safe."

31. gaiacom doctor and gaiacom version

31.1 Doctor

```
sudo /opt/gaiacom/gaiacom-backend doctor \
--config /etc/gaiacom/gaiacom.env
```

Doctor checks, among other things:

- Configuration syntax
- Secret strength/formats
- Server identity
- Database directory
- File permissions
- Production-critical settings

On Unix, the production environment file must not have unnecessary group/world permissions.

31.2 Version

```
/opt/gaiacom/gaiacom-backend version
```

The output includes release/protocol/build information such as version, commit and build time.

32. Governance Bootstrap

A governance configuration is intended for the initial node operator.

Example:

```
{  
  "bootstrap_gaia_id": "@operator:node.example.org"  
}
```

Save as:

```
/etc/gaiacom/governance.json
```

Set restrictive permissions:

```
sudo chown root:root /etc/gaiacom/governance.json  
sudo chmod 600 /etc/gaiacom/governance.json
```

Add the configuration path to the environment:

```
echo 'GAIACOM_GOVERNANCE_CONFIG=/etc/gaiacom/governance.json' \  
| sudo tee -a /etc/gaiacom/gaiacom.env >/dev/null
```

After startup, the bootstrap GaiaID must actually exist as a suitable identity or be brought through the intended bootstrap flow.

33. Systemd Hardening

The repository provides a hardened `deploy/systemd/gaiacom.service`.

Included or intended protections include:

- unprivileged user `gaiacom`
- `NoNewPrivileges=true`
- `PrivateTmp=true`
- `PrivateDevices=true`
- `ProtectSystem=strict`
- `ProtectHome=true`
- restrictive capability boundaries
- `LockPersonality=true`
- `RestrictSUIDSGID=true`
- `UMask=0077`

After installation:

```
sudo systemctl daemon-reload
sudo systemctl enable --now gaiacom
sudo systemctl status gaiacom
```

Logs:

```
sudo journalctl -u gaiacom -f
```

34. NGINX, TLS and Reverse Proxy

34.1 Routing

NGINX should:

- serve `/` from the built frontend,
- proxy `/api/` to `127.0.0.1:8080`,
- forward `/livez` and `/readyz` internally,
- forward `/.well-known/gaiacom/` to the backend,
- not expose `/metrics` publicly.

34.2 Security headers

The supplied vhost specifies strict headers including, in essence:

- Content-Security-Policy
- X-Frame-Options: `DENY`
- X-Content-Type-Options: `nosniff`

The CSP must not be casually weakened through broad `*`, `unsafe-inline` or additional third-party origins, because the web client in particular is part of the cryptographic trust path.

34.3 TLS

Set up a TLS certificate with Certbot or equivalent infrastructure and monitor automatic renewal.

35. Firewall and Network Exposure

Minimal public exposure:

- SSH, if required
- TCP 80 for redirect/ACME
- TCP 443 for GaiaCom HTTPS

Example UFW:

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

Do not expose publicly: backend port `8080`, unless an intentionally different and separately hardened operating model is being used.

36. Node Registry and Controlled Federation

36.1 Prerequisites

A node requires:

- valid FQDN
- DNS
- HTTPS
- server signing key
- correct Well-Known endpoints
- registry/authority configuration
- valid release/core information

36.2 Well-Known endpoints

Central federation discovery/transport paths include:

```
GET /.well-known/gaiacom/server
GET /.well-known/gaiacom/nodeinfo
GET /.well-known/gaiacom/nodes
POST /.well-known/gaiacom/s2s/v1/forward
```

36.3 Joining

Typical flow:

1. Deploy the node.
2. Run `setup` and `doctor` successfully.
3. Enable the service and TLS.
4. Provide the bootstrap operator GaiaID.
5. Ping the main node/registry from Nodesystem.
6. Check admission/quarantine/block status.
7. Send a federated test message.

36.4 Controlled federation

Sovereign Beta v2 is **not a freely open federation**. Nodes are admitted in a controlled manner.

36.5 S2S security

The federation path contains protections against:

- invalid signatures
- replay

- invalid time windows
 - body-hash manipulation
 - incorrect target
 - SSRF to private/disallowed networks
-

37. Storage: Local Object Store and S3/MinIO

37.1 Local Object Store

By default, backend storage can keep encrypted chunks in the local file system.

Configuration-relevant variables:

```
GAIACOM_STORAGE_ROOT  
GAIACOM_STORAGE_USER_QUOTA_BYTES  
GAIACOM_STORAGE_PENDING_TTL_HOURS
```

The Object Store code contains path-jail/path-validation logic. Custom adapters must not bypass this protection layer.

37.2 S3/MinIO

Optional:

```
GAIACOM_OBJECT_STORE=s3  
GAIACOM_S3_ENDPOINT  
GAIACOM_S3_BUCKET  
GAIACOM_S3_REGION  
GAIACOM_S3_ACCESS_KEY  
GAIACOM_S3_SECRET_KEY  
GAIACOM_S3_PREFIX  
GAIACOM_S3_PATH_STYLE
```

Even with S3, the Object Store should only receive chunk data that has already been encrypted client-side.

37.3 Access models

Storage metadata supports owner/grant/public-style ACL states as well as expiration/revocation paths.

"Public" does not mean that a client-side encrypted object automatically becomes publicly available in decrypted form; access and cryptographic models must be considered together.

38. Configuring SMTP for Production

Relevant variables in the repository/documentation state:

```
GAIACOM_SMTP_HOST
GAIACOM_SMTP_PORT
GAIACOM_SMTP_FROM
GAIACOM_SMTP_USERNAME
GAIACOM_SMTP_PASSWORD
GAIACOM_SMTP_INGEST_TOKEN
GAIACOM_SMTP_TLS_MODE
```

A typical port is 587 for STARTTLS, depending on the relay.

Production check:

1. Relay authenticated?
 2. TLS enforced?
 3. `From` domain correct?
 4. SPF configured?
 5. DKIM signed?
 6. DMARC active?
 7. Bounce/abuse behavior monitored?
 8. Ingest token strong and secret?
 9. SMTP endpoint rate-limited?
 10. Downgrade visible in the UI?
-

39. Health Checks, Metrics and Logs

39.1 Liveness

```
curl --fail --silent https://node.example.org/livez
```

39.2 Readiness

```
curl --fail --silent https://node.example.org/readyz
```

39.3 Nodeinfo

```
curl --fail --silent \  
https://node.example.org/.well-known/gaiacom/nodeinfo
```

39.4 Nodes

```
curl -s https://node.example.org/.well-known/gaiacom/nodes
```

39.5 Public Version

`/api/v1/public/version` should reflect the version, commit ID and build time actually built.

39.6 Metrics

In the intended setup, `/metrics` is **not public**. Locally or through authorized monitoring access, the endpoint requires the Metrics token. When the token is absent or invalid, the protection path should not disclose unnecessary details.

39.7 Journald

```
sudo journalctl -u gaiacom --since today  
sudo journalctl -u gaiacom -f
```

Secrets or plaintext key material must not end up in logs.

40. Backups and Recovery

40.1 SQLite

Before updates, create a consistent SQLite backup, for example using the SQLite backup function or `.backup`.

Example:

```
sudo systemctl stop gaiacom
sqlite3 /var/lib/gaiacom/gaiacom.db \
    ".backup '/var/backups/gaiacom/gaiacom-$(date +%F-%H%M%S).db'"
sudo systemctl start gaiacom
```

40.2 Migration snapshots

The internal migration manager creates backups before pending schema migrations. These are **not a substitute for a complete backup strategy**.

40.3 Object Store

The database and Object Store must be backed up in a coordinated manner. A database copy without the corresponding chunk objects may be incomplete.

40.4 Secrets

Back up production secrets separately and in encrypted form:

- Environment
- Governance configuration
- Required server-side identity keys

Loss of server-identity or trust-related secrets can have federation/verification consequences.

40.5 Client Recovery

A server backup does not replace user recovery. Private client keys/mnemonics are deliberately not stored on the server as a plaintext Godmode backup.

41. Updates and Rollback

41.1 Before the update

- Review release notes.
- Run tests/gates in the build.
- Back up the database.
- Back up the Object Store.
- Back up secrets/configuration.
- Archive the current binary/frontend build.

41.2 Update

1. Stop the service.
2. Install the new backend binary.
3. Deploy the new frontend atomically or in a controlled manner.
4. Start the service.
5. Check `livez`, `readyz`, `nodeinfo`, version and federation.
6. Inspect logs for migration/runtime errors.

41.3 Rollback

A binary rollback is only safe if the older binary is compatible with the now-migrated database schema.

If the migration is incompatible, restore the **previous database backup together with the matching Object Store state**.

42. Production Checklist

Before public operation:

- ☐ FQDN and DNS correct.
 - ☐ TLS valid and auto-renewal tested.
 - ☐ Backend bound only to loopback.
 - ☐ `GAIACOM_ALLOW_PUBLIC_BIND=false` unless a deliberately different model is used.
 - ☐ `gaiacom setup` successful.
 - ☐ `gaiacom doctor` without critical errors.
 - ☐ Configuration file mode `0600`.
 - ☐ Secrets strong and not stored in Git.
 - ☐ Governance bootstrap correct.
 - ☐ systemd hardening active.
 - ☐ NGINX hardening active.
 - ☐ `/metrics` not externally reachable.
 - ☐ `livez / readyz` successful.
 - ☐ Version/commit/build time correct.
 - ☐ SQLite/Object Store backup tested.
 - ☐ SMTP TLS/SPF/DKIM/DMARC checked if enabled.
 - ☐ Federation tested in a controlled manner.
 - ☐ Browser CSP not weakened.
 - ☐ Security/hygiene gates successful.
 - ☐ AGPLv3 licensing consistent across `LICENSE`, README, package metadata and source notices.
 - ☐ Unreleased beta features not advertised as production-ready.
-

Part III - Developer Manual

43. Repository Structure

Simplified:

```
GaiaCom-main/
├─ Frontend/frontend/      React 18 / Vite Client
├─ Backend/                Go Backend
├─ SovereignCryptoCore/    Rust Crypto Core
├─ SovereignCryptoWasm/    WASM Bindings
├─ DesktopClient/          Tauri 2 Desktop Shell
├─ Core/rust/gaiacore/     Zero-/Low-Dependency Core Primitives
├─ Android/                Kotlin/Compose modular mobile platform
├─ deploy/                 NGINX + systemd
├─ security/               PoCs / adversarial tests
├─ scripts/                Hygiene / Architecture Gates
├─ docs/                   Operations, Security, Federation, Runbooks
├─ README.md
├─ SECURITY.md
├─ SECURITY_INVARIANTS.md
└─ ARCHITECTURE.md         short repository architecture file
```

The separately supplied large architecture map is significantly more comprehensive and is included in full in the appendix to this manual.

44. Frontend Architecture

44.1 Stack

- React 18
- Vite
- Lazy Loading / Suspense
- Web Worker
- WebAssembly
- OPFS
- PWA Service Worker
- local i18n layer

44.2 Central entry points

```
Frontend/frontend/src/index.jsx  
Frontend/frontend/src/App.jsx  
Frontend/frontend/src/api.js  
Frontend/frontend/src/crypto.js  
Frontend/frontend/src/sovereignCrypto.js
```

44.3 UI orchestration

`App.jsx` manages global UI/session state. `AppMainContent` loads the active pane. `AppModalLayer` centralizes modal/dialog surfaces.

44.4 Important panes

- DashboardPane
- ChatPane
- GroupChatPane
- PublicChannelsPane
- ReaderPane
- ComposerPane
- ProfilePane
- DrivePane
- DropPane
- GsnPane
- SecurityCenter
- AbuseCenter
- NetworkHealthDashboard

44.5 Hooks

Domain logic is encapsulated, among other places, in:

- `useGaiaAuth`
- `useChat`
- `useEmails`
- `useDrive`
- `useGaiaDrop`
- `usePublicChannels`
- `useGsn`
- `usePresence`
- `useKeyChangeDetection`
- `useInactivityLock`

44.6 Crypto Provider

`sovereignCrypto.js` abstracts between:

- Web provider with isolated Crypto Worker/WASM
- Native provider through Tauri IPC

The provider path must not silently degrade when required cryptographic primitives are unavailable.

45. Backend Architecture

45.1 Stack

- Go
- `net/http` through custom `Backend/httpx`
- pure-Go SQLite
- domain-oriented packages
- no Gin/GORM in the current architecture path

45.2 Entrypoint

```
Backend/cmd/gaiacom/main.go
```

delegates into the backend lifecycle.

45.3 Main domains

```
auth/  
devices/  
identity/  
messaging/  
mailbox/  
room/  
publicchannels/  
gsn/  
storage/  
gaiaDROP/  
smtpbridge/  
federation/  
noderegistry/  
trustmesh/  
governance/  
internal/security/  
networkhealth/  
operations/  
provision/  
mobileapi/
```

45.4 Security middleware

`httpx`, GaiaShield and domain-specific authorization checks form multiple layers. BOLA/BFLA protection must not exist only in the frontend.

45.5 Scheduler

`scheduler.go` processes scheduled draft/send actions.

46. Persistence and SQLite

46.1 Database mode

The database layer uses pure-Go SQLite and configures, among other things:

- WAL
- Foreign Keys
- Busy Timeout
- `synchronous=FULL`

46.2 Repository Layer

`Backend/repository` provides Store interfaces and SQL implementations.

Domain adapters cover, among other things:

- Auth
- Devices
- Identity
- Messaging
- Mailbox
- Rooms
- Public Channels
- GSN
- Storage/GaiaDrop
- Federation
- Governance
- Security Events
- Rate Limits
- Presence
- Offline Transport Queue

46.3 Migrations

Migrations use a ledger/checksum model and create database snapshots before pending migrations.

46.4 Scaling boundary

Beta operation is designed around **one backend process per SQLite database**. SQLite in this design is not an active multi-primary/horizontally scaled database cluster.

47. Sovereign Crypto Core, WASM and Desktop

47.1 Hybrid KEM

The design combines:

- ephemeral X25519
- ML-KEM-1024
- HQC-256

through a shared key derivation with SHA3/HKDF binding.

47.2 Accelerated Cascade

Design:

- Twofish-256-EAX
- AES-256-GCM
- Ed25519 signature

47.3 Top Secret Cascade

Design:

- Serpent-256-CTR + HMAC-SHA3-512
- Twofish-256-EAX
- XChaCha20-Poly1305
- AES-256-GCM-SIV
- Ed25519
- ML-DSA-87

47.4 SovereignCryptoCore

Rust implementation of the cipher cascades and controlled memory zeroization.

47.5 WASM

`SovereignCryptoWasm` exports the Rust primitives for the web client. Browser HQC additionally runs through the local isolated provider/worker path.

47.6 Desktop Tauri

The Tauri host intentionally exposes a narrow IPC surface:

```
sovereign_hqc256_keypair
sovereign_hqc256_encapsulate
```

```
sovereign_hqc256_decapsulate  
sovereign_seal  
sovereign_open
```

The narrow bridge reduces the number of native functions that must be reachable from the webview.

47.7 Desktop Custom Node

In the current snapshot, the desktop API origin is pinned to <https://beta.gaiacom.de>. For a custom node, both origin logic and the Tauri CSP must be reviewed, adjusted and rebuilt.

48. Android Platform

The Android structure is modular:

```
:app
:platform:contracts
:platform:kernel
:platform:node
:platform:security
:platform:security-android
:platform:security-auth
:platform:runtime
:platform:remote
:platform:api
:platform:identity
```

Planned/implemented foundations include:

- Jetpack Compose UI
- Embedded Go Node
- Android Keystore
- StrongBox
- Biometrics/PIN
- typed API facade
- Wi-Fi Direct/BLE transport-adapter foundations

Status: technical beta. According to the release boundaries, the complete Mobile Sovereign v1 message path must not yet be treated as fully production-ready.

49. API Overview

The complete endpoint reference is contained in the architecture map in Appendix A. For day-to-day operation, the following domain overview is sufficient.

49.1 Auth

```
/api/v1/auth/register  
/api/v1/auth/login  
/api/v1/auth/refresh  
/api/v1/auth/logout  
/api/v1/auth/status  
/api/v1/auth/change-password  
/api/v1/auth/delete-account
```

49.2 Devices

Device sessions, device keys and pairings under `/api/v1/devices/...`

49.3 Identity

Identity creation, own identity, public identity, Human Proof and Trust Passport.

49.4 Messaging

Send, Inbox, Read, Edit, Delete, Reactions, Proofs and Presence.

49.5 Mailbox

Message state, Drafts, Labels, Filters, Mailbox Settings and global search.

49.6 Rooms

Rooms, members, channels, join requests, invitations and moderation.

49.7 Public Channels

Channels, posts, comments, reactions, subscriptions and moderation.

49.8 GSN

Feed, Posts, Comments, Reactions, Profiles and Follows.

49.9 Storage

Upload Init, Chunks, Complete, Download, Grants and metadata.

49.10 GaiaDrop

Public submit path plus authenticated Inbox/Read/Delete functions.

49.11 SMTP

Outbound Send and protected Ingest.

49.12 Governance / Reports

Reports, Reviewer, Node Abuse, Disclosure and Gaia's Eyes.

49.13 Security / Node

Security Events, Node Security, Registry and operator actions.

49.14 Public / Operations

Network Health, Version, Transparency, `livez`, `readyz`, protected Metrics.

49.15 Federation

Well-Known discovery and signed S2S forwarding.

50. Security and Quality Gates

The README release path documents a gate run across backend, frontend, Rust, WASM, Tauri and repository hygiene.

Example:

```
cd Backend
go mod verify
go test ./...

cd ../Frontend/frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund
npm test
npm run build
npm run test:hqc-browser

cd ../../SovereignCryptoCore
cargo test --locked

cd ../SovereignCryptoWasm
cargo test --locked
wasm-pack build --target web --release

cd ../DesktopClient/src-tauri
cargo test --locked

cd ../../
python scripts/repository_hygiene.py
```

Additional suites exist:

```
security/master_security_suite.py
security/total_system_poc_runner.py
security/poc/
security/extreme/
scripts/android_architecture_gate.py
```

PoC/adversarial areas cover, among other things:

- Federation SSRF
- Replay
- Signature verification
- GaiaDrop size/XSS/rate-limit behavior
- GaiaProof
- Room Authorization
- Governance
- Key History
- Secure Disclosure
- SMTP Injection/Downgrade
- Trust Passport
- BOLA/BFLA

- Database/concurrency stress

Note: These commands are the release gate documented in the repository. This manual does not claim that every toolchain check was fully rerun during documentation production.

51. Developing and Extending Securely

51.1 Classify a new feature first

Before implementation, ask:

- Does it touch plaintext?
- Does it touch private keys?
- Does it touch envelopes?
- Does it touch federation?
- Does it touch authorization?
- Does it touch disclosure/governance?
- Does it touch object storage?
- Does it add an external origin or dependency?

The more questions are answered with yes, the stronger the security review must be.

51.2 Do not implement security only in the UI

A hidden button is not access control.

Authorization must be checked server-side for the specific object and the specific action.

51.3 No silent downgrade

If Top Secret, a PQC capability or a recipient key is missing:

- Make the error visible.
- Do not transparently switch to weaker security.

51.4 Treat CSP as a security boundary

In the web client in particular, the delivered JavaScript application is part of the trust path. New CDN dependencies, inline scripts or third-party origins change the threat model.

51.5 Minimize logs

Never place the following in debug/audit logs:

- Private keys
- Mnemonics
- Recovery secrets
- JWT secrets
- SMTP passwords
- Plaintext messages

51.6 Migrations

Schema changes should be:

- migration-based
- auditable
- backwards-/rollback-aware
- performed with backups
- tested under concurrent access

51.7 New federation paths

Additionally test:

- SSRF
 - private IP ranges
 - DNS rebinding
 - replay
 - clock skew
 - body hash
 - signature
 - target confusion
 - queue poisoning
 - resource exhaustion
-

Part IV - Operations Support

52. Troubleshooting

52.1 Frontend cannot reach the backend locally

Symptoms: CORS errors, login fails immediately, API "network error."

Check:

```
VITE_API_URL=http://localhost:8080
Frontend at http://localhost:3000
Backend at 127.0.0.1:8080
GAIACOM_DEV_MODE=true
```

Do not accidentally mix `127.0.0.1:5173` and `localhost:3000` when the CORS allowlist contains only part of those origins.

52.2 `.env` exists but seems to be ignored

The backend does not automatically treat a file as a `.env` loader application.

Solution:

```
set -a
source /path/to/gaiacom.env
set +a
./gaiacom-backend
```

In production, systemd loads `EnvironmentFile=` from the supplied unit.

52.3 Production server does not start because of build info

Create the production build with version, commit and build time. Do not simply copy an unqualified development binary.

52.4 `doctor` complains about file permissions

```
sudo chown root:root /etc/gaiacom/gaiacom.env
sudo chmod 600 /etc/gaiacom/gaiacom.env
```

52.5 `readyz` fails

Check:

- Is the database reachable?
- Did migrations complete successfully?
- Is the storage directory writable?

- Is the runtime configuration valid?
- Logs: `journalctl -u gaiacom`

52.6 `/metrics` returns 404

This may be intentional. Metrics are protected; a missing/incorrect bearer token should not publicly reveal the endpoint. NGINX should block it externally anyway.

52.7 Federation works locally but not between nodes

Check:

- DNS
- TLS
- Server Name
- Registry status
- Node admission
- Well-Known endpoints
- System time
- Signing key
- SSRF/address policy
- Target node supports the required security suite
- Quarantine/block status

52.8 Top Secret cannot be enabled

This is correct when a required capability is missing.

Check:

- Recipient keyset
- HQC provider
- ML-KEM key
- Required signature keys
- Client provider
- Key-change warning
- Node/protocol capability

Do not "fix" the issue by removing the check.

52.9 Desktop build always talks to `beta.gaiacom.de`

The current snapshot pins the desktop origin. Review/adjust `Frontend/frontend/src/api.js` and the Tauri CSP, then rebuild.

52.10 Registration is possible despite `OPEN_REGISTRATION=false`

In the reviewed snapshot, this variable is not wired as an unambiguous gatekeeper in the user-registration service.

If closed user registration is required, the authentication endpoint must be explicitly gated server-side and tested.

52.11 Chat shows attachment functionality while the release says "disabled"

The UI contains experimental paths. The release matrix governs Sovereign Beta v2. Disable unreleased attachment/media paths or mark them visibly as experimental.

52.12 Migration succeeded, but the rollback binary will not start

The schema may now be incompatible. Restore the matching database backup together with the corresponding Object Store state.

52.13 SMTP works, but messages land in spam

Check:

- PTR/rDNS
 - SPF
 - DKIM
 - DMARC
 - From alignment
 - Relay reputation
 - TLS
 - Rate
 - Content
 - Bounce handling
-

53. Known Limitations of Sovereign Beta v2

1. **SQLite deployment:** One backend process per database; no active multi-primary HA database.
2. **Web-origin trust:** A compromised node could theoretically deliver manipulated JavaScript to the web client before a message is encrypted.
3. **Endpoint compromise:** Unlocked devices and compromised operating systems remain outside the core cryptographic assumption.
4. **No anonymity guarantee:** Content encryption does not eliminate all metadata.
5. **SMTP downgrade:** Traditional email lies outside GaiaCom's full security model.
6. **Attachments:** Chat/media attachments are not yet fully released.
7. **GaiaDrop:** The release path is text-only; attachments are not yet released.
8. **Mobile:** Android architecture exists, but the complete Sovereign v1 mobile message path remains a technical beta.
9. **Federation:** Controlled, not open.
10. **Governance:** Cross-node policies must not be interpreted as a global consensus.
11. **GaiaDrive Cross Device:** The key-share/recovery ceremony remains a beta boundary.
12. **Automatic Updates:** No fully released automated update pipeline.
13. **Recovery Operations:** Recovery processes must be rehearsed organizationally.
14. **Accessibility/low-end performance:** Part of the release gates, but still a beta quality area.

GaiaCom should therefore not be marketed as "unhackable," "fully anonymous," "high availability without additional architecture," or "immune to compromised endpoints/servers."

54. Glossary

AAD - Additional Authenticated Data; metadata that can be cryptographically bound to ciphertext.

BFLA - Broken Function Level Authorization.

BOLA - Broken Object Level Authorization.

CSP - Content Security Policy.

E2EE - End-to-End Encryption.

Envelope - Structured encrypted message carrier.

Federation - Communication between independent GaiaCom nodes.

GaiaID - Federated GaiaCom address.

GaiaProof - Cryptographic proof format.

GaiaShield - Security/audit subsystem.

Gaia's Eyes - Controlled disclosure/reviewer workflow.

GSN - GaiaSocialNetwork.

HQC-256 - Post-quantum-oriented KEM in the Sovereign design.

ML-DSA-87 - Post-quantum-oriented signature scheme.

ML-KEM-1024 - Post-quantum-oriented Key Encapsulation Mechanism.

OPFS - Origin Private File System in the browser.

PDU - Protocol Data Unit in federation transport.

PQC - Post-Quantum Cryptography.

S2S - Server-to-Server.

TrustMesh - Local/policy-based reputation system.

Trust Passport - Aggregated identity/trust view.

WASM - WebAssembly.

Zero Server Trust - Design principle in which confidential content does not need to be decrypted by the server.

55. Documentation Maintenance

At every relevant release change, at least the following documents should be reviewed together:

```
README.md
SECURITY.md
SECURITY_INVARIANTS.md
DEPLOYMENT.md
docs/*
Frontend/frontend/package.json
Backend/go.mod
Backend/provision/*
deploy/nginx/*
deploy/systemd/*
```

The large architecture map should also be updated whenever any of the following change:

- Crypto suites
- API endpoints
- Tables
- Roles
- Modules
- Frontend panes
- Federation flows
- Deployment requirements

For future releases, a clear documentation hierarchy is recommended:

1. **Source code + tests** - actual behavior.
2. **Release Contract / README** - released functions and boundaries.
3. **Deployment Guide** - canonical production operation.
4. **Architecture** - in-depth technical description.
5. **Manual** - user- and operations-oriented consolidation.

Appendix A - Complete Architecture Map (English Translation)

The following section is a complete English translation of the separately supplied detailed architecture document. File paths, endpoint paths, identifiers, code symbols and cryptographic primitive names are retained exactly where they function as technical identifiers.

GaiaCom Technical System and Architecture Map

// STATUS: PLATIN VGT SUPREME

// REVISION: 2.0.0 Sovereign Beta v2

// ARCHITECTURE DOCUMENTATION & SYSTEMATIC DATA TREE

Contents (Architecture Index)

1. System Overview

- Architecture Philosophy & Zero Server Trust
- Runtime Environments & Technology Stack
- Global Architecture Tree

2. Mapping Files to the Architecture

- Frontend Client (React / Vite)
- Desktop Shell (Tauri 2 Native Client)
- Sovereign Crypto Core & WebAssembly
- Core Rust Engine (Zero Dependency)
- Backend Application Layer (Go net/http & httpx)
- Persistence & SQLite Database
- Android Embedded Node & Modular Platform
- Deployment, DevOps & Hardening
- Security PoCs, Audits & Quality Gates

3. Module Documentation

- Module 1: Sovereign Cryptographic Engine (Dual-PQC & Multi-Cipher Cascade)
- Module 2: Authentication, Sessions & Key Vault (GaiaVault / Auth)
- Module 3: Identity, GaiaID & Trust Passport
- Module 4: Quantum Chat & E2EE Direct Messaging
- Module 5: Group Chats, Rooms & Channels
- Module 6: GaiaMail & Mailbox State Management
- Module 7: SMTP Legacy Bridge & Downgrade Gateway
- Module 8: Public Channels
- Module 9: GSN - GaiaSocialNetwork
- Module 10: GaiaDrive & Encrypted Chunk Storage
- Module 11: GaiaDrop
- Module 12: Federation & S2S PDU Transport
- Module 13: Node Registry & Core-Hash Verification
- Module 14: TrustMesh & Local Reputation Scoring
- Module 15: Governance, Gaia's Eyes & Reviewer Workflows
- Module 16: GaiaShield Security Operations & Intrusion Defense
- Module 17: Desktop Client Integration (Tauri 2 Bridge)
- Module 18: Android Mobile Platform (Embedded Node)

4. Detailed Dashboard & Workspace Documentation

5. CSS & UI Architecture

6. API Architecture & Endpoint Reference

- 7. System-Wide Data Flows
 - 8. Shared & Core Components
 - 9. Architecture Relations & Mermaid Diagrams
 - 10. File References
 - 11. Architecture Observations & Unused Files
-

1. System Overview

1.1 Architecture Philosophy & Zero Server Trust

GaiaCom is a **post-quantum-secure, federated communications infrastructure** built around strict principles of **Zero Server Trust** and **Client-Side Computing**:

- 1. Zero Server Trust:** The server (Go backend) acts only as a transport carrier, routing instance, federation node and storage layer for encrypted envelopes (`message_envelopes`), encrypted chunks (`file_chunks`) and minimal identity/audit metadata. At no time does the server have access to:
 - Plaintext messages or attachments
 - BIP-39 mnemonics or private keys (X25519, ML-KEM-1024, HQC-256, Ed25519, ML-DSA-87)
 - Symmetric message keys or session root secrets
 - Decrypted contents of local storage (GaiaDrive / GaiaVault)
- 2. Dual-PQC Hybrid KEM:** Encryption is based on a three-secret combiner (`HKDF-SHA3-512`) that combines classic ephemeral X25519, post-quantum ML-KEM-1024 and post-quantum HQC-256.
- 3. Multi-Cipher Authenticated Cascade:**
 - *Accelerated Mode*: Twofish-256-EAX + AES-256-GCM (Ed25519 signatures)
 - *Top Secret Mode*: Four-stage cascade: Serpent-256-CTR-HMAC-SHA3-512 + Twofish-256-EAX + XChaCha20-Poly1305 + AES-256-GCM-SIV (Ed25519 + ML-DSA-87 signatures)
- 4. No administrative master key (No Godmode):** Neither node operators nor auditors can retrospectively decrypt encrypted data. Moderation is proof-based through cryptographic proofs (`GaiaProof`), disclosure tokens (`Gaia's Eyes`) and local federation reputation (`TrustMesh`).
- 5. Zero-Dependency Edge:** Gin and GORM have been fully removed from the backend. HTTP routing is handled through `Backend/httpx` (pure Go `net/http`), and persistence through pure-Go SQLite (`modernc.org/sqlite`).

1.2 Runtime Environments & Technology Stack

Layer	Technology	Purpose	Source Path
Web Frontend	React 18, Vite, Web Worker	Single-page application, UI, cryptographic browser worker, OPFS	Frontend/frontend/
Desktop Shell	Tauri 2.11, Rust 1.85	Native desktop app, isolated IPC, native HQC and cascade commands	DesktopClient/src-tauri/
Sovereign Crypto Core	Rust 1.85, zeroize	Shared multi-cipher cascade, native & WASM compilation	SovereignCryptoCore/
Sovereign Crypto WASM	Rust, wasm-pack	WebAssembly bridge for the browser crypto worker	SovereignCryptoWasm/
Core Primitives	Rust (Zero Dependency)	Deterministic validation (GaiaID, Envelope, TrustMesh)	Core/rust/gaiacore/
Backend Server	Go 1.26.5 (Linux AMD64)	Federation, routing, SQLite persistence, ACLs, GaiaShield, SMTP	Backend/
In-Memory Mobile Node	Go (Backend/mobileapi)	Embedded node for Android/iOS through Gomobile bridge	Backend/mobileapi/

Layer	Technology	Purpose	Source Path
Android Client	Kotlin, Jetpack Compose	Modular native mobile node with Keystore/StrongBox	Android/
Database	SQLite (pure-Go WAL mode)	Local and server-side relational persistence with triggers	Backend/database/
Edge Proxy	NGINX	TLS termination, CSP headers, reverse proxy (:8080)	deploy/nginx/
Service Supervisor	systemd	Sandboxed Linux service with minimal privileges	deploy/systemd/

1.3 Global Architecture Tree

GAIACOM PLATFORM

- └─ Dashboard & Workspace (Frontend UI) [Frontend/frontend/src/]
 - └─ Layout & Navigation
 - └─ NavigationSidebar.jsx (Main navigation: Workspace, Network, Personal)
 - └─ ListPane.jsx (List view: mailbox, rooms, contacts, GaiaDrop, GaiaDrive)
 - └─ MobileNavigationDock.jsx (Mobile bottom dock for touch devices)
 - └─ LogoMark.jsx (SVG branding and AstraeaOS icon)
 - └─ VersionBadge.jsx (Build and consensus indicator)
 - └─ Dashboard (Command Center)
 - └─ DashboardPane.jsx (System metrics, uplink status, PWA install, quick actions)
 - └─ Workspace Panes (Communication Cores)
 - └─ ChatPane.jsx (Quantum Chat: 1-to-1 E2EE PQC messages)
 - └─ GroupChatPane.jsx (Group chats: rooms, subchannels, moderation)
 - └─ PublicChannelsPane.jsx (Public Channels: public posts, comments, reactions)
 - └─ ReaderPane.jsx (Mail reader: native and SMTP mail display)
 - └─ ComposerPane.jsx (Mail composer: draft management, scheduling, SMTP warning)
 - └─ Network, Security & Governance Panes
 - └─ GsnPane.jsx (GaiaSocialNetwork: decentralized feed, profiles, reactions)
 - └─ NetworkHealthDashboard.jsx (Network health, ping latency, federation nodes)
 - └─ SecurityCenter.jsx (GaiaShield: security events, audit chain, node audit)
 - └─ AbuseCenter.jsx (Reporting center: case reports, Gaia's Eyes, reviewer queue)
 - └─ Personal Areas & Storage
 - └─ ProfilePane.jsx (Profile management, mnemonics, keys, inactivity lock)
 - └─ DrivePane.jsx (GaiaDrive: OPFS-encrypted files, local notes, cloud sync)
 - └─ DropPane.jsx (GaiaDrop: anonymous text submissions & inbox)
 - └─ VaultPane.jsx (Legacy GaiaVault notes; superseded by DrivePane)
 - └─ Modal & Dialog Layers
 - └─ AppModalLayer.jsx (Central modal manager)
 - └─ AppFeedbackModals.jsx (User confirmations, alerts, 3-way dialogs)
 - └─ AuthScreen.jsx (Login, registration, account recovery)
 - └─ UnlockScreen.jsx (Session unlock via password / PIN / WebAuthn)
 - └─ SetupWizard.jsx (First-setup wizard for new identities)
 - └─ FirstRunOnboarding.jsx (Introductory tour after first installation)
 - └─ ContactProfileModal.jsx (Contact detail card, Trust Passport, key history)
 - └─ AddContactModal.jsx (Manual contact import via GaiaID)
 - └─ CreateGroupModal.jsx (Room creation with Top Secret option)
 - └─ JoinGroupModal.jsx (Join room via hash or invitation link)
 - └─ CreateChannelModal.jsx (Create subchannel in an existing room)
 - └─ GroupSettingsModal.jsx (Room settings, Slow Mode, member management)
 - └─ KeyChangeWarningModal.jsx (Security warning for changed recipient key)
 - └─ QuantumShieldModal.jsx (Explanation & status display of quantum protection)
 - └─ HumanProofDialog.jsx (Cryptographic proof-of-human evidence)
 - └─ Styling Architecture [Frontend/frontend/src/styles/]
 - └─ 27 cascading CSS files (base, theme, layout, chat, nebula, ui, etc.)
 - └─ startup.css (Critical pre-hydration styles to avoid FOUCC)
- └─ Sovereign Cryptographic Engine (Dual-PQC & Multi-Cipher)
 - └─ SovereignCryptoCore/ (Rust cascade: Twofish, AES-GCM, Serpent, XChaCha20, AES-GCM-SIV)
 - └─ SovereignCryptoWasm/ (WASM export for web browsers)
 - └─ DesktopClient/src-tauri/ (Tauri 2 native host with pqcrypto-hqc & IPC bridge)

- └─ Core/rust/gaiacore/ (Zero-dependency Rust primitives: GaiaID, Envelopes, TrustMesh)
- └─ Frontend/frontend/src/sovereign/
 - └─ nativeProvider.js (Desktop Tauri IPC Provider)
 - └─ webProvider.js (Browser Worker & WASM Provider)
 - └─ webCrypto.worker.js (Isolated Web Worker for HQC-256 & cascade)
 - └─ vendor/liboqs-hqc256/ (Local, vendor-independent liboqs HQC-256 runtime)
- └─ Frontend/frontend/src/crypto.js (Classic & PQC primitives: noble-curves, scure-bip39, AES-GCM)
- └─ Backend/crypto/ (Go CIRCL ML-KEM-1024, ML-DSA-87, Ed25519)
- └─ Backend Application Services (Go 1.26.5) [Backend/]
 - └─ cmd/gaiacom/main.go (CLI entry point: run, setup, doctor, version)
 - └─ main.go (Server lifecycle, graceful shutdown, signal handling)
 - └─ routes.go (HTTP router setup, middleware cascade, route declarations)
 - └─ route_runtime.go (Runtime configuration validation)
 - └─ scheduler.go (DraftScheduler: scheduled sending of mail/messages)
 - └─ httpx/ (Framework-free net/http routing, CORS & security headers)
 - └─ auth/ (User accounts, Argon2/PBKDF2, JWT signing & rotation)
 - └─ devices/ (Device sessions, key revocation, QR pairing protocol)
 - └─ identity/ (GaiaID management, public key catalogs, HumanProof)
 - └─ messaging/ (E2EE message transport, proofs, read receipts, reactions)
 - └─ mailbox/ (Mailbox state, folders, labels, contacts, filters, search)
 - └─ room/ (Room management, member roles, join requests, pins, audit logs)
 - └─ publicchannels/ (Public broadcasts, subscriptions, moderation, comments)
 - └─ gsn/ (Social network: posts, comments, reactions, profiles)
 - └─ storage/ (GaiaDrive Chunked Object Store: Local FS & S3 adapter, quotas)
 - └─ gaiadrop/ (Anonymous text submissions with rate limiting & hash validation)
 - └─ smtpbridge/ (Legacy SMTP inbound & outbound with downgrade labeling)
 - └─ federation/ (S2S Signed PDUs, SSRF firewall, replay protection, queue worker)
 - └─ noderegistry/ (Node registration, operator admission, core-hash integrity)
 - └─ trustmesh/ (Local reputation assessment, epoch hashing, abuse friction)
 - └─ governance/ (Bootstrap roles, reporting cases, Gaia's Eyes disclosure, appeals)
 - └─ internal/security/ (GaiaShield: cryptographic audit chain, redaction filters, IDS rules)
 - └─ networkhealth/ (Node metrics, latency pings, dashboard data)
 - └─ operations/ (Health checks: /livez, /readyz, /metrics Prometheus monitoring)
 - └─ provision/ (Idempotent production initialization & environment diagnostics)
 - └─ mobileapi/ (In-process Gomobile bindings for native Android/iOS nodes)
- └─ Persistence & Database Layer [Backend/database/ & Backend/repository/]
 - └─ database/database.go (SQLite connection manager, WAL pragmas, DDL migrations)
 - └─ database/migrations.go (Schema migration ledger with SHA-256 checksums & backups)
 - └─ database/migration_transport_queue.go (Offline transport outbox and deduplication tables)
 - └─ repository/repository.go (Central Store interface definitions)
 - └─ repository/sql_store.go (SQLStore constructor and transaction management)
 - └─ repository/sql_store_*.go (25 domain-specific persistence adapters)
- └─ Android Mobile Platform [Android/]
 - └─ platform/contracts/ (Interface contracts between modules)
 - └─ platform/kernel/ (App lifecycle, command bus, event dispatcher)
 - └─ platform/node/ (Bridge to Go embedded node via Backend/mobileapi)
 - └─ platform/security/ (Cryptographic interfaces & GaiaCore Rust-JNI)
 - └─ platform/security-android/ (Android Keystore & StrongBox hardware binding)
 - └─ platform/security-auth/ (Biometric authentication & PIN lock)
 - └─ platform/runtime/ (Local runtime management)
 - └─ platform/remote/ (Transport adapters for HTTP, Wi-Fi Direct, BLE)
 - └─ platform/api/ (Typed Kotlin APIs for presentation layer)
 - └─ platform/identity/ (Identity and key management on device)
 - └─ app/ (Android Jetpack Compose UI entry point)
- └─ Deployment, DevOps & Runtime Hardening [deploy/]
 - └─ nginx/gaiacom.conf (Hardened NGINX vhost: TLS 1.3, CSP, Frame Deny, No Sniff)
 - └─ systemd/gaiacom.service (Hardened systemd unit: strict sandbox, PrivateTmp, NoNewPrivileges)
- └─ Test & Security Gates [security/ & scripts/]
 - └─ scripts/repository_hygiene.py (Hygiene scanner: checks binary hashes, file sizes & secrets)
 - └─ scripts/android_architecture_gate.py (Validates Android module dependencies)
 - └─ security/master_security_suite.py (Automated end-to-end adversarial suite)
 - └─ security/poc/ (11 isolated proof-of-concept exploit & integrity tests)
 - └─ security/extreme/ (Extreme load, BOLA/BFLA and replay stress tests)

2. Mapping Files to the Architecture

All paths are relative to the repository root (`GaiaCom-main/`).

2.1 Frontend Client (`Frontend/frontend/`)

Entry points & configuration

- `Frontend/frontend/index.html` : HTML root document with Content Security Policy (CSP), PWA manifest links and startup CSS.
- `Frontend/frontend/vite.config.js` : Vite bundler configuration, WASM support, Rollup bundle splitting.
- `Frontend/frontend/package.json` : Node package definition (React 18, noble-curves, scure-bip39, vitest).
- `Frontend/frontend/src/index.jsx` : React 18 root initialization, `RuntimeBoundary` error containment, PWA service-worker registration.
- `Frontend/frontend/src/index.css` : Main CSS manifest; imports all 27 individual interface stylesheets.
- `Frontend/frontend/public/startup.css` : Critical fallback styling before React hydration to prevent FOUC.
- `Frontend/frontend/public/service-worker.js` : Offline caching and asset management for Progressive Web Apps.

Global components & orchestration

- `Frontend/frontend/src/App.jsx` : Main container; manages theme, identities, menu state, notifications and socket/polling behavior.
- `Frontend/frontend/src/components/app/AppMainContent.jsx` : Dynamic viewport; loads the active pane using `React.lazy` and `Suspense`.
- `Frontend/frontend/src/components/app/AppModalLayer.jsx` : Central layer for popups, modals and dialogs.
- `Frontend/frontend/src/components/app/AppFeedbackModals.jsx` : Alert, confirmation and three-way selection dialogs.
- `Frontend/frontend/src/api.js` : Central HTTP API client with automatic JWT rotation and desktop detection.
- `Frontend/frontend/src/crypto.js` : Web cryptography engine (noble/curves, scure/bip39, PBKDF2, AES-GCM, ECDSA).
- `Frontend/frontend/src/sovereignCrypto.js` : Abstraction layer for Sovereign Dual-PQC, switching between native and Web Worker providers.

Navigation & layout

- `Frontend/frontend/src/components/layout/NavigationSidebar.jsx` : Three-area sidebar (Workspace, Network, Personal), identity card and Quantum Shield indicator.
- `Frontend/frontend/src/components/layout/ListPane.jsx` : Middle column for message lists, chat contacts, folders and filters.

- `Frontend/frontend/src/components/layout/MobileNavigationDock.jsx` : Touch-optimized navigation for phones and tablets.
- `Frontend/frontend/src/components/layout/LogoMark.jsx` : Vector-based GaiaCom logo.
- `Frontend/frontend/src/components/layout/VersionBadge.jsx` : Displays software version and federation consensus.

Dashboard & feature panes

- `Frontend/frontend/src/components/chat/DashboardPane.jsx` : System command center, uplink status and quick overview.
- `Frontend/frontend/src/components/chat/ChatPane.jsx` : E2EE one-to-one Quantum Chat, timeline, file-attachment UI and reactions.
- `Frontend/frontend/src/components/chat/GroupChatPane.jsx` : Group-room view, subchannels, participant list and role display.
- `Frontend/frontend/src/components/chat/PublicChannelsPane.jsx` : Public broadcast channels, posts, reactions and comments.
- `Frontend/frontend/src/components/chat/ReaderPane.jsx` : Mail reader, native-envelope decoding and GaiaProof verification.
- `Frontend/frontend/src/components/chat/ComposerPane.jsx` : Mail editor with subject, recipients, draft storage and SMTP switch.
- `Frontend/frontend/src/components/chat/ProfilePane.jsx` : Key display, mnemonics, inactivity timer, account deletion and privacy.
- `Frontend/frontend/src/components/chat/DrivePane.jsx` : GaiaDrive storage manager (OPFS files, notes, cloud sync).
- `Frontend/frontend/src/components/chat/DropPane.jsx` : GaiaDrop submission inbox and review of anonymous submissions.
- `Frontend/frontend/src/components/chat/GsnPane.jsx` : GaiaSocialNetwork feed, post creation, follow/unfollow.
- `Frontend/frontend/src/components/chat/SecurityCenter.jsx` : GaiaShield security center, event logs and node audit.
- `Frontend/frontend/src/components/chat/AbuseCenter.jsx` : Reporting center, trust assessment, reviewer queue and operator actions.
- `Frontend/frontend/src/components/public/NetworkHealthDashboard.jsx` : Public network status, ping times and federation health.
- `Frontend/frontend/src/components/chat/VaultPane.jsx` : *Legacy*: old note vault, superseded by `DrivePane.jsx`.

Authentication & onboarding

- `Frontend/frontend/src/components/auth/AuthScreen.jsx` : Authentication screen (login, registration, recovery).
- `Frontend/frontend/src/components/auth/UnlockScreen.jsx` : Local session unlock (password, PIN, WebAuthn PRF).
- `Frontend/frontend/src/components/auth/SetupWizard.jsx` : Wizard for generating new identities and mnemonics.

- `Frontend/frontend/src/components/onboarding/FirstRunOnboarding.jsx` : Interactive introduction after initial setup.

Modals & dialogs

- `Frontend/frontend/src/components/modals/AddContactModal.jsx` : Add contact by GaiaID.
- `Frontend/frontend/src/components/modals/ContactProfileModal.jsx` : Contact profile, verification evidence and key history.
- `Frontend/frontend/src/components/modals/CreateGroupModal.jsx` : Create a group room with Top Secret flag.
- `Frontend/frontend/src/components/modals/JoinGroupModal.jsx` : Join a group through invitation code.
- `Frontend/frontend/src/components/modals/CreateChannelModal.jsx` : Create a channel in a room.
- `Frontend/frontend/src/components/chat/GroupSettingsModal.jsx` : Room administration, Slow Mode, role permissions and audit logs.
- `Frontend/frontend/src/components/modals/KeyChangeWarningModal.jsx` : Security warning for a changed recipient key.
- `Frontend/frontend/src/components/modals/QuantumShieldModal.jsx` : Details of Dual-PQC protection status.
- `Frontend/frontend/src/components/common/HumanProofDialog.jsx` : Creates a cryptographic Human Proof.
- `Frontend/frontend/src/components/common/AppModal.jsx` : Generic modal container.
- `Frontend/frontend/src/components/common/GaiaPassportCard.jsx` : Visual passport card containing trust values.
- `Frontend/frontend/src/components/common/AvatarPicker.jsx` : Avatar selection and upload preparation.

Frontend Custom Hooks (`Frontend/frontend/src/hooks/`)

- `useGaiaAuth.js` : Authentication state, token management and identity retrieval.
- `useChat.js` : Chat messages, polling, sending, decryption, deletion and reactions.
- `useDrive.jsx` : OPFS file management, local AES-GCM notes and chunked cloud uploads.
- `useEmails.js` : GaiaMail and SMTP mailbox logic, filters, drafts, labels and state updates.
- `usePublicChannels.js` : Subscribe to public channels, create posts and manage comments.
- `useGsn.js` : Retrieve GSN feeds, publish posts, reactions and profile changes.
- `useGaiaDrop.js` : Retrieve and clean up GaiaDrop submissions.
- `usePresence.js` : Send heartbeats and query contacts' online state.
- `useMessageMeta.js` : Pins, saved messages and reactions in local storage.
- `useUnreadMarkers.js` : Unread markers and position tracking for chats and groups.
- `useChatNotifications.js` : Desktop and audio notifications for new messages.
- `useInactivityLock.js` : Automatically locks the UI on inactivity.
- `useKeyChangeDetection.js` : Detects attempted manipulation of recipient keys.
- `useVault.js` : *Legacy*: old vault hook, superseded by `useDrive.jsx`.

Frontend utilities (`Frontend/frontend/src/utils/`)

- `avatar.js` : Avatar generation and scaling.
- `channelsTranslations.js` : Multilingual system text for Public Channels.
- `contacts.js` : Merges mail and chat contacts.
- `deviceFanout.js` : Fanout calculation for multi-device encryption.
- `gaiaAddress.js` : Parsing and validation of `@user:node.tld` addresses.
- `humanProof.js` : Proof-of-Humanity calculations.
- `i18n.jsx` : Internationalization engine (DE, EN, ES, FR, IT, etc.).
- `keyHistory.js` : Append-only tracking of verified contact keys.
- `markdown.jsx` : Safe React rendering of Markdown without `dangerouslySetInnerHTML`.
- `notificationPreferences.js` : Normalizes notification settings.
- `payload.js` : Validates and serializes message payloads.
- `recovery.js` : Export and import of encrypted recovery files.
- `safeJson.js` : Hardened JSON parsing against prototype pollution.
- `secureExport.js` : Encrypted export of system data.
- `terms.js` : Terms of use and privacy statements.
- `useContactActions.js` : Helper functions for contact management.
- `useProfileActions.js` : Helper functions for profile and key management.
- `uuid.js` : RFC 4122 v4 UUID generation and validation.
- `webauthnPrf.js` : WebAuthn PRF extension for hardware-bound key derivation.

2.2 Desktop Shell (`DesktopClient/`)

- `DesktopClient/src-tauri/Cargo.toml` : Tauri project configuration and dependencies (`pqcrypto-hqc`, `zeroize`, `gaiacom-sovereign-core`).
- `DesktopClient/src-tauri/tauri.conf.json` : Security boundaries: local web server disabled, strict CSP, window properties.
- `DesktopClient/src-tauri/capabilities/default.json` : Fine-grained Tauri 2 capability declaration.
- `DesktopClient/src-tauri/src/main.rs` : Desktop execution entry point.
- `DesktopClient/src-tauri/src/lib.rs` : Registers native IPC commands in the Tauri builder.
- `DesktopClient/src-tauri/src/sovereign_crypto.rs` : Implements five isolated IPC crypto commands:
 - `sovereign_hqc256_keypair` : Generates an HQC-256 key pair natively in Rust.
 - `sovereign_hqc256_encapsulate` : Produces KEM ciphertext and shared secret.
 - `sovereign_hqc256_decapsulate` : Decapsulates KEM ciphertext with the private key.
 - `sovereign_seal` : Executes Twofish/AES-GCM or the four-stage cascade natively.
 - `sovereign_open` : Decrypts cascade ciphertext natively in Rust.

2.3 Sovereign Crypto Core & WebAssembly

- `SovereignCryptoCore/Cargo.toml`: Rust crate `gaiacom-sovereign-core` v2.0.0.
 - `SovereignCryptoCore/src/lib.rs`: Mathematical implementation of the cipher cascades:
 - `seal()` & `open()`: Main functions for Accelerated and Top Secret profiles.
 - Twofish-256-EAX, AES-256-GCM, Serpent-256-CTR, HMAC-SHA3-512, XChaCha20-Poly1305, AES-256-GCM-SIV.
 - Secure zeroization (`zeroize`) of all temporary key buffers.
 - `SovereignCryptoWasm/Cargo.toml`: Crate for WASM compilation using `wasm-pack`.
 - `SovereignCryptoWasm/src/lib.rs`: WebAssembly bindings (`#[wasm_bindgen]`) that expose the Rust cascade to browsers.
 - `Frontend/frontend/src/sovereign/wasm/gaiacom_sovereign_wasm_bg.wasm`: Compiled WASM binary.
 - `Frontend/frontend/src/sovereign/vendor/liboqs-hqc256/hqc-256.runtime.js`: Local liboqs HQC-256 JavaScript/WASM runtime.
 - `Frontend/frontend/src/sovereign/webCrypto.worker.js`: Isolated Web Worker that executes HQC-256 operations and the cascade in a background thread.
-

2.4 Core Rust Engine (`Core/rust/gaiacore/`)

- `Core/rust/gaiacore/Cargo.toml`: Zero-dependency crate `gaiacore`.
 - `Core/rust/gaiacore/src/lib.rs`: Fundamental invariants:
 - `validate_gaia_id()`: RFC-compliant validation of `@user:domain.tld`.
 - `constant_time_eq()`: Timing-resistant byte comparison.
 - `is_fixed_hex()`: Safe hexadecimal validation.
 - `Core/rust/gaiacore/src/envelope.rs`: Parsing and integrity validation of message envelopes.
 - `Core/rust/gaiacore/src/trust_mesh.rs`: Mathematical calculation of reputation scores and decay curves.
-

2.5 Backend Application Layer (`Backend/`)

Main program & routing

- `Backend/cmd/gaiacom/main.go`: Binary entry point; delegates to `backend.Execute()`.
- `Backend/main.go`: Server lifecycle, initialization of configuration, database and routes, graceful shutdown.
- `Backend/routes.go`: Registers all endpoints, middleware pipelines and background workers.
- `Backend/route_runtime.go`: Loads and validates environment variables (`GAIACOM_JWT_SECRET`, etc.).
- `Backend/scheduler.go`: Background service for delayed/scheduled mail and messages.
- `Backend/embedded_node.go`: Enables in-process operation as an embedded engine.
- `Backend/mobile_node_registry.go`: Manages active node instances in mobile contexts.
- `Backend/mobile_transport_registry.go`: Registers available mobile transport paths.

Domain packages

- **Backend/auth/ :**
 - `auth_service.go` : User registration, Argon2 password hashing, JWT issuance.
 - `auth_handler.go` : HTTP handlers for `/api/v1/auth/*`.
- **Backend/devices/ :**
 - `device_service.go` : Device-key management, revocation and QR-code pairing.
 - `device_handler.go` : HTTP handlers for `/api/v1/devices/*`.
- **Backend/identity/ :**
 - `identity_service.go` : CRUD for identities, publishing of public keys.
 - `identity_handler.go` : HTTP handlers for `/api/v1/identity/*` and public profiles.
- **Backend/messaging/ :**
 - `messaging_service.go` : Envelope validation, E2EE routing and proof generation.
 - `messaging_handler.go` : HTTP handlers for `/api/v1/messaging/*`.
- **Backend/mailbox/ :**
 - `service.go` : Mailbox folders, labels, filter rules, address book and global full-text search.
 - `handler.go` : HTTP handlers for `/api/v1/mailbox/*` and `/api/v1/search/global`.
- **Backend/room/ :**
 - `service.go` : Group rooms, subchannels, roles, invitation links and join requests.
 - `handler.go` : HTTP handlers for `/api/v1/rooms/*`.
- **Backend/publicchannels/ :**
 - `service.go` : Public broadcast channels, posts, comments, reactions and moderation.
 - `handler.go` : HTTP handlers for `/api/v1/public-channels/*`.
- **Backend/gsn/ :**
 - `service.go` : GaiaSocialNetwork posts, reactions, comments, profiles and followers.
 - `handler.go` : HTTP handlers for `/api/v1/gsn/*`.
- **Backend/storage/ :**
 - `storage_service.go` : Chunked upload coordination, quotas and access permissions (ACL).
 - `storage_handler.go` : HTTP handlers for `/api/v1/storage/*`.
 - `object_store.go` : File-system-based local object store with path-jail protection.
 - `s3_object_store.go` : S3/MinIO-compatible object-store adapter.
- **Backend/gaiadrop/ :**
 - `service.go` : Anonymous text submissions, hash validation and size limiting.
 - `handler.go` : HTTP handlers for `/api/v1/gaiadrop/*` and `/api/v1/public/gaiadrop/*`.
- **Backend/smtpbridge/ :**
 - `service.go` : Legacy SMTP gateway with STARTTLS, header sanitization and downgrade labeling.
 - `handler.go` : HTTP handlers for `/api/v1/smtp/send` and `/api/v1/public/smtp/ingest`.

- `Backend/federation/` :
 - `federation_service.go` : Server-to-server communication, signed PDUs and replay protection.
 - `federation_handler.go` : HTTP handlers for `/.well-known/gaiacom/*` and S2S forwarding.
 - `federation_types.go` : Data structures for federation transactions.
- `Backend/noderegistry/` :
 - `service.go` : Registration of new nodes, operator admission, quarantine and core-hash comparison.
 - `handler.go` : HTTP handlers for `/api/v1/node/registry/*`.
- `Backend/trustmesh/` :
 - `service.go` : Proof-based abuse reports, reputation reduction and epoch hashing.
 - `handler.go` : HTTP handler for `/api/v1/reports/submit`.
- `Backend/governance/` :
 - `service.go` : Bootstrap operator rights (`governance.json`), Gaia's Eyes tokens and reviewer queue.
 - `handler.go` : HTTP handlers for `/api/v1/governance/*`, `/api/v1/reviewer/*`, `/api/v1/node/abuse/*`.
 - `bootstrap.go` : Loads trusted operator identities at system startup.
- `Backend/internal/security/` (GaiaShield):
 - `shield.go` : Central security subsystem, EdgeShield middleware, retention sweeper.
 - `audit.go` : Tamper-resistant cryptographic audit chain (hash chain).
 - `events.go` : Defines and records security events (severity, categories).
 - `privacy.go` : Automatic masking and anonymization of sensitive data.
 - `security_routes.go` : HTTP handlers for `/api/v1/security/*` and `/api/v1/node/security/*`.
- `Backend/networkhealth/` :
 - `service.go` : Uptime measurement, latency checks and neighboring-node status.
 - `handler.go` : HTTP handler for `/api/v1/public/network-health`.
- `Backend/operations/` :
 - `monitor.go` : Liveness (`/livez`), readiness (`/readyz`) and Prometheus metrics (`/metrics`).
- `Backend/httpx/` :
 - `router.go` : Zero-dependency HTTP router with parameterized paths.
 - `middleware.go` : Security headers (HSTS, CSP, framing protection).
 - `cors.go` : Fine-grained CORS policy.
 - `json.go` : Standardized JSON output and error serialization.
- `Backend/crypto/` :
 - `crypto_service.go` : CIRCL ML-KEM-1024, Ed25519 and ML-DSA-87 helpers.
- `Backend/models/` :
 - `models.go` : All Go structs for entities, API requests, DTOs and database rows.

- `Backend/buildinfo/` :
 - `buildinfo.go` : Immutable release metadata (version, Git commit, build time).
- `Backend/provision/` :
 - `setup.go` : Idempotent production-environment generation (`gaiacom setup`).
 - `doctor.go` : Automated security validation of configuration (`gaiacom doctor`).

2.6 Persistence & SQLite Database

- `Backend/database/database.go` :
 - SQLite initialization (`modernnc.org/sqlite` , pure-Go, CGO-free).
 - Enforces WAL mode (`PRAGMA journal_mode=WAL`), foreign keys (`PRAGMA foreign_keys=ON`), busy timeout (10,000 ms) and FULL synchronous level.
 - Contains the DDL migration table and more than 70 DDL statements.
- `Backend/database/migrations.go` :
 - Deterministic schema migration manager.
 - Performs an atomic SQLite backup before each pending migration (`backupDatabaseBeforeMigration`).
- `Backend/database/migration_transport_queue.go` :
 - Schema for the offline transport outbox (`transport_outbox`) and deduplication (`transport_inbox_dedup`).
- `Backend/repository/repository.go` :
 - Interfaces: `Store` , `AuthStore` , `MessagingStore` , `StorageStore` , `GovernanceStore` , etc.
- `Backend/repository/sql_store.go` :
 - Main `SQLStore` adapter, wrapping `*sql.DB` and transaction helpers.
- `Backend/repository/sql_store_*.go` :
 - 25 domain-specific implementation files:
 - `sql_store_auth.go` : Users, sessions, passwords.
 - `sql_store_devices.go` / `sql_store_device_keys.go` : Devices and pairings.
 - `sql_store_identity.go` : Identities, Human Proofs.
 - `sql_store_messaging.go` : Envelopes, inboxes, proofs, read receipts, reactions.
 - `sql_store_mailbox.go` : Mailbox states, drafts, labels, filters, search.
 - `sql_store_rooms_public_channels.go` : Rooms, channels, memberships, moderation.
 - `sql_store_public_channel_interactions.go` : Public Channel posts, reactions, comments.
 - `sql_store_gsn.go` : Posts, comments, reactions, profiles, followers.
 - `sql_store_storage_gaiadrop.go` : File metadata, chunks, access rights, Drop submissions.
 - `sql_store_federation.go` : Server list, queue, S2S routing.
 - `sql_store_federation_replay.go` : Replay checks for incoming PDUs.
 - `sql_store_governance_abuse.go` : Policies, role credentials, cases, disclosures.

- `sql_store_security_events.go` : GaiaShield security events & audit chain.
 - `sql_store_rate_limit.go` : Rate-limit counters in SQLite.
 - `sql_store_presence.go` : Heartbeat and online states.
 - `sql_store_transport_queue.go` : Multi-hop offline queue.
-

2.7 Android Embedded Node & Modular Platform (Android/)

- `Android/settings.gradle.kts` : Gradle multi-module structure:
 - `:app` : Native Jetpack Compose client.
 - `:platform:contracts` : Interfaces for decoupling all feature modules.
 - `:platform:kernel` : Application kernel, command/event bus and state management.
 - `:platform:node` : In-process JNI bridge to the Go backend (`Backend/mobileapi`).
 - `:platform:security` : Abstract security and cryptographic interfaces.
 - `:platform:security-android` : Hardware-backed key storage in Android Keystore / StrongBox.
 - `:platform:security-auth` : Biometric authentication (fingerprint / face unlock).
 - `:platform:runtime` : Lifecycle and background-service control.
 - `:platform:remote` : Transport connections (Internet, Wi-Fi Direct, Bluetooth Mesh).
 - `:platform:api` : Typed Kotlin facade for presentation layers.
 - `:platform:identity` : Local identity and address-book management.
 - `Android/build.gradle.kts` : Central build configuration.
-

2.8 Deployment, DevOps & Hardening (deploy/)

- `deploy/nginx/gaiacom.conf` :
 - Hardened NGINX reverse proxy.
 - Forwards `/api/v1/` , `/.well-known/gaiacom/` , `/livez` and `/readyz` to local backend port `:8080` .
 - Blocks direct external access to `/metrics` .
 - Protects static frontend files with strict headers: `X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , `Content-Security-Policy` .
 - `deploy/systemd/gaiacom.service` :
 - Hardened systemd service unit for Linux AMD64.
 - `ProtectSystem=strict` , `ProtectHome=true` , `PrivateTmp=true` , `PrivateDevices=true` .
 - `NoNewPrivileges=true` , `LockPersonality=true` , `RestrictSUIDSGID=true` .
 - Unprivileged execution under system user `gaiacom:gaiacom` with `UMask=0077` .
-

2.9 Security PoCs, Audits & Quality Gates (security/ & scripts/)

- `scripts/repository_hygiene.py` : Prevents committing private keys, unauthorized binaries, logs or passwords; scans 653+ files.

- `scripts/android_architecture_gate.py` : Verifies strict compliance with Android module boundaries.
 - `security/master_security_suite.py` : Executes the full automated security validation suite.
 - `security/total_system_poc_runner.py` : Orchestrates system-wide PoC attack scenarios.
 - `security/poc/` :
 - `poc/federation/` : Tests SSRF protection, IP blocks, invalid signatures and replay attacks.
 - `poc/gaiadrop/` : Tests payload size limits, XSS filtering and rate limits.
 - `poc/gaiaproof/` : Verifies tamper resistance of GaiaProof certificates.
 - `poc/gaiarooms-pro/` : Tests room-access permissions, role escalation and Top Secret integrity.
 - `poc/gaiavault/` : Verifies that client-side-encrypted records remain unreadable on the server.
 - `poc/governance/` : Tests role credentials, signature chains and threshold consensus.
 - `poc/key-history/` : Checks detection of manipulated recipient keys (downgrade protection).
 - `poc/secure-disclosure/` : Tests time-limited disclosure using Gaia's Eyes tokens.
 - `poc/smtp-bridge/` : Tests protection against CRLF injection and strict downgrade signaling.
 - `poc/total-system/` : Full-system hardening test.
 - `poc/trust-passport/` : Validates reputation metrics against forgery.
 - `security/extreme/` : Extreme stress tests for concurrent database writes and BOLA/BFLA attacks.
-

3. Module Documentation

Module 1: Sovereign Cryptographic Engine (Dual-PQC & Multi-Cipher Cascade)

Purpose

This module forms GaiaCom's fundamental cryptographic foundation. It provides confidentiality, authenticity and post-quantum security for messages, files and identities.

Capabilities

- **Dual-PQC Hybrid KEM:** Combines ephemeral X25519 (classical), ML-KEM-1024 (FIPS 203) and HQC-256 (NIST PQC Round 4) through HKDF-SHA3-512 into a shared root secret.
- **Sovereign Accelerated Cascade:** Authenticated encryption using Twofish-256-EAX and AES-256-GCM, signed with Ed25519.
- **Sovereign Top Secret Cascade:** Four-stage authenticated cascade: Serpent-256-CTR-HMAC-SHA3-512 + Twofish-256-EAX + XChaCha20-Poly1305 + AES-256-GCM-SIV; dual-signed with Ed25519 and ML-DSA-87 (FIPS 204).
- **Transcript Binding:** Cryptographically binds all metadata (version, sender, recipient, message ID, timestamp, algorithm suite) to the ciphertext using AAD.
- **Fail-Closed Provider Architecture:** Seamlessly switches between the Tauri native host and isolated Web Worker; blocks immediately when required primitives are missing.

Associated files

```
Frontend:
- Frontend/frontend/src/sovereignCrypto.js
- Frontend/frontend/src/crypto.js
- Frontend/frontend/src/sovereign/webProvider.js
- Frontend/frontend/src/sovereign/nativeProvider.js
- Frontend/frontend/src/sovereign/webCrypto.worker.js
- Frontend/frontend/src/sovereign/wasm/gaiacom_sovereign_wasm.js
- Frontend/frontend/src/sovereign/vendor/liboqs-hqc256/hqc-256.runtime.js

Desktop Shell:
- DesktopClient/src-tauri/src/sovereign_crypto.rs
- DesktopClient/src-tauri/src/lib.rs

Sovereign Core (Rust):
- SovereignCryptoCore/src/lib.rs
- SovereignCryptoWasm/src/lib.rs

Core Primitives (Rust):
- Core/rust/gaiacore/src/lib.rs
- Core/rust/gaiacore/src/envelope.rs

Backend:
- Backend/crypto/crypto_service.go
- Backend/crypto/kyber.go
- Backend/crypto/types/crypto_types.go

Tests:
- Frontend/frontend/src/adversarial.test.js
- Frontend/frontend/src/hqc-smoke.js
- SovereignCryptoCore/tests / DesktopClient/src-tauri/tests
```

Dashboard integration

- **Used in:** `ChatPane.jsx`, `GroupChatPane.jsx`, `ComposerPane.jsx`, `ReaderPane.jsx`, `DrivePane.jsx`.
- **Visual indicators:** `QuantumShieldModal.jsx`, `HQC-256 + ML-KEM-1024 VERIFIED` badge in `DashboardPane.jsx`, security status in `NavigationSidebar.jsx`.
- **User actions:** Switch between *Accelerated* and *Top Secret* mode in chat; display the cryptographic verification certificate.

Dependencies

```
Sovereign Cryptographic Engine
├─ requires WebAssembly & Web Worker runtime in the browser
├─ requires pqcrypto-hqc & gaiacom-sovereign-core in the desktop client
├─ requires cloudflare/circl in the backend (ML-KEM / ML-DSA)
└─ exports primitives to Messaging, Storage, GSN and Drive
```

Used by

- Module 2 (Authentication & Key Vault)
- Module 4 (Quantum Chat)
- Module 5 (Group Chats)
- Module 6 (GaiaMail)
- Module 9 (GSN Social)
- Module 10 (GaiaDrive)

Module 2: Authentication, Sessions & Key Vault (GaiaVault / Auth)

Purpose

Manages user accounts, secure client-side generation and safekeeping of keys (Key Vault), session tokens (JWT access and refresh tokens) and pairing of new devices using QR codes.

Capabilities

- Account registration with Argon2id password hashing on the server and PBKDF2/BIP-39 key derivation in the client.
- Secure session management through short-lived JWT access tokens and long-lived refresh tokens with automatic rotation.
- Device pairing through ephemeral one-time secrets without transferring private keys.
- Inactivity lock with local unlock using password, PIN or WebAuthn PRF hardware token.

Associated files

```
Frontend:
- Frontend/frontend/src/components/auth/AuthScreen.jsx
- Frontend/frontend/src/components/auth/UnlockScreen.jsx
- Frontend/frontend/src/components/auth/SetupWizard.jsx
- Frontend/frontend/src/hooks/useGaiaAuth.js
- Frontend/frontend/src/hooks/useInactivityLock.js
- Frontend/frontend/src/utils/webauthnPrf.js
```

```
- Frontend/frontend/src/utils/recovery.js
```

Backend:

```
- Backend/auth/auth_service.go
- Backend/auth/auth_handler.go
- Backend/devices/device_service.go
- Backend/devices/device_handler.go
```

API:

```
- POST /api/v1/auth/register, /login, /refresh, /logout, /change-password, /delete-account
- GET /api/v1/auth/status, /devices
- POST /api/v1/devices/pairings, /devices/pairings/:id/approve, /consume
```

Database:

```
- users, identities, device_sessions, device_pairings, device_keys
```

Dashboard integration

- After login, entry into `DashboardPane.jsx`.
- Unlock through `UnlockScreen.jsx` after inactivity.
- Session management in `ProfilePane.jsx`, including viewing and revoking active devices.

Dependencies

Authentication & Vault

```
├─ uses Module 1 (Sovereign Crypto) for key derivation
├─ reads/writes tables: users, identities, device_sessions, device_keys
├─ uses shared components: api.js, safeJson.js
```

Used by

- All modules in the overall system as an access prerequisite.

Module 3: Identity, GaiaID & Trust Passport

Purpose

Enables decentralized addressing of communication partners in the form `@username:node.domain.tld`, provides cryptographic identity evidence and manages the Trust Passport.

Capabilities

- Parsing and validation of federated GaiaIDs (`gaiaAddress.js` & `Core/rust/gaiacore/src/lib.rs`).
- Management of public key catalogs (X25519 box key, ML-KEM-1024 key, HQC-256 key, Ed25519 signing key, ML-DSA-87 signing key).
- HumanProof verification: cryptographic evidence of human interaction.
- Trust Passport: aggregated trust and reputation metrics for a user.
- Key Change Detection: warning when a recipient key changes to help defend against man-in-the-middle attacks.

Associated files

Frontend:

```
- Frontend/frontend/src/components/common/GaiaPassportCard.jsx
- Frontend/frontend/src/components/common/HumanProofDialog.jsx
```

```

- Frontend/frontend/src/components/modals/ContactProfileModal.jsx
- Frontend/frontend/src/components/modals/KeyChangeWarningModal.jsx
- Frontend/frontend/src/hooks/useKeyChangeDetection.js
- Frontend/frontend/src/utls/gaiaAddress.js
- Frontend/frontend/src/utls/humanProof.js
- Frontend/frontend/src/utls/keyHistory.js

Backend:
- Backend/identity/identity_service.go
- Backend/identity/identity_handler.go

API:
- POST /api/v1/identity/create, /human-proof
- GET /api/v1/identity/me, /public/identity/:gaiaID, /public/trust-passport/:gaiaID

Database:
- identities, user_notification_preferences

```

Dashboard integration

- Active identity displayed in `NavigationSidebar.jsx` and `DashboardPane.jsx`.
- Address-book detail view in `ContactProfileModal.jsx`.

Dependencies

```

Identity & Trust Passport
├─ requires Backend/identity & Core/rust/gaiacore
├─ writes the identities table
└─ provides identity information to all communication modules

```

Used by

- Module 4 (Quantum Chat), Module 5 (Groups), Module 6 (GaiaMail), Module 9 (GSN), Module 15 (Governance).

Module 4: Quantum Chat & E2EE Direct Messaging

Purpose

Provides direct end-to-end-encrypted one-to-one communication with Dual-PQC key exchange, read receipts, real-time presence and reactions.

Capabilities

- Post-quantum-secure encryption of every chat message on the sending device.
- Server stores only unreadable envelopes (`message_envelopes`) and delivery evidence.
- Supports *Accelerated* and *Top Secret* security levels.
- Real-time presence detection (online status and typing indicators).
- Later editing (`/messaging/edit`), deletion (`/messaging/delete`) and reactions (`/messaging/reaction`).
- Generation of `GaiaProof` certificates for evidentiary use in abuse cases.

Associated files

Frontend:

- Frontend/frontend/src/components/chat/ChatPane.jsx
- Frontend/frontend/src/components/chat/MessageActions.jsx
- Frontend/frontend/src/hooks/useChat.js
- Frontend/frontend/src/hooks/usePresence.js
- Frontend/frontend/src/hooks/useMessageMeta.js
- Frontend/frontend/src/hooks/useUnreadMarkers.js

Backend:

- Backend/messaging/messaging_service.go
- Backend/messaging/messaging_handler.go
- Backend/presence/service.go
- Backend/presence/handler.go

API:

- POST /api/v1/messaging/send, /read, /edit, /reaction, /delete, /clear
- GET /api/v1/messaging/inbox, /proof
- POST /api/v1/presence/heartbeat, /typing; GET /presence/status, /typing

Database:

- message_envelopes, inboxes, message_proofs, delivery_receipts, message_read_receipts, message_reactions, identity_presence

Dashboard integration

- `chat` menu item opens `ChatPane.jsx`.
- Unread direct-message count appears in `NavigationSidebar.jsx`.
- Quick access to recent conversations in `DashboardPane.jsx`.

Dependencies

Quantum Chat

- └─ uses Module 1 (Sovereign Crypto) for KEM & cascades
- └─ uses Module 3 (Identity) for recipient keys
- └─ reads/writes `message_envelopes`, `inboxes`, `message_proofs`
- └─ integrates GaiaDrive for file sharing

Used by

- End users for confidential direct communication.

Module 5: Group Chats, Rooms & Channels

Purpose

Provides multi-user group rooms with role-based access control, subchannels, join requests, invitation links and moderation logs.

Capabilities

- Rooms with multiple topic-oriented text channels (`channels`).
- Role hierarchy: `owner`, `admin`, `moderator`, `member`.
- Private and public rooms with join requests (`room_join_requests`).
- Time-limited and usage-limited invitation links (`room_invite_links`).
- Pinned messages (`room_pinned_messages`) and Slow Mode throttling.

- Immutable revision and moderation logs (`room_moderation_logs`).

Associated files

Frontend:

- Frontend/frontend/src/components/chat/GroupChatPane.jsx
- Frontend/frontend/src/components/chat/GroupSettingsModal.jsx
- Frontend/frontend/src/components/modals/CreateGroupModal.jsx
- Frontend/frontend/src/components/modals/JoinGroupModal.jsx
- Frontend/frontend/src/components/modals/CreateChannelModal.jsx

Backend:

- Backend/room/service.go
- Backend/room/handler.go

API:

- POST /api/v1/rooms/create, /update, /join, /leave, /delete, /channels, /members/role, /members/kick, /transfer-ownership
- POST /api/v1/rooms/pins/toggle, /invites/create, /invites/join, /join-requests/create, /join-requests/moderate
- GET /api/v1/rooms, /channels, /search, /pins, /join-requests, /moderation-logs

Database:

- rooms, room_members, channels, room_pinned_messages, room_invite_links, room_join_requests, room_moderation_logs

Dashboard integration

- `groups` menu item loads `GroupChatPane.jsx`.
- Room overview and quick join in `DashboardPane.jsx`.

Dependencies

Group Chats

- └─ uses Module 4 (Messaging) for message envelopes
- └─ reads/writes tables: rooms, room_members, channels
- └─ interacts with Module 16 (GaiaShield) for moderation actions

Module 6: GaiaMail & Mailbox State Management

Purpose

Implements a full client-side-encrypted mail system (GaiaMail) with folders, labels, server-side scheduling and global search.

Capabilities

- Folder management: Inbox, Drafts, Sent, Starred, Important, Snoozed, Archive, Spam, Trash.
- Custom colored labels (`mail_labels`).
- Scheduled mail delivery through `DraftScheduler`.
- Automated filter rules (`mail_filter_rules`) based on sender and subject.
- Privacy-conscious global search (`/api/v1/search/global`).

Associated files

Frontend:

- Frontend/frontend/src/components/layout/ListPane.jsx (Mailbox mode)
- Frontend/frontend/src/components/chat/ReaderPane.jsx

```

- Frontend/frontend/src/components/chat/ComposerPane.jsx
- Frontend/frontend/src/hooks/useEmails.js

Backend:
- Backend/mailbox/service.go
- Backend/mailbox/handler.go
- Backend/scheduler.go

API:
- GET /api/v1/mailbox/messages, /drafts, /labels, /contacts, /filters, /settings, /search/global
- POST /api/v1/mailbox/state, /drafts/save, /drafts/delete, /labels/save, /contacts/save, /filters/save, /settings

Database:
- mailbox_states, mail_drafts, mail_labels, mail_contacts, mail_filter_rules, mail_settings

```

Dashboard integration

- Opened through the `GaiaMail` dropdown in `NavigationSidebar.jsx`.
- Latest unread messages are displayed in the `DashboardPane.jsx` widget.

Module 7: SMTP Legacy Bridge & Downgrade Gateway

Purpose

Enables exchange with traditional email systems over SMTP while explicitly warning about the loss of native GaiaCom security guarantees.

Capabilities

- Outbound email through STARTTLS with strict CRLF and header-injection protection.
- Inbound webhook `/api/v1/public/smtp/ingest` with token authentication.
- Explicitly marks all SMTP messages as `untrusted` in the frontend.
- Persistent warning in the Composer when SMTP mode is selected.

Associated files

```

Frontend:
- Frontend/frontend/src/components/layout/NavigationSidebar.jsx (SMTP dropdown)
- Frontend/frontend/src/components/chat/ComposerPane.jsx (SMTP mode & warning)

Backend:
- Backend/smtplib/service.go
- Backend/smtplib/handler.go

API:
- POST /api/v1/smtp/send
- POST /api/v1/public/smtp/ingest

```

Dashboard integration

- Dedicated `SMTP Mail` navigation area in `NavigationSidebar.jsx`.
- Visual differentiation through warning badges (`smtp-badge`).

Module 8: Public Channels

Purpose

Provides open, subscribable information channels for announcements, discussion threads, media and community updates.

Capabilities

- Creation of public channels with name, description, avatar and categories.
- Subscribe and unsubscribe.
- Publication of posts with formatting, media attachments and scheduling (`scheduled_for`).
- Emoji reactions and multi-level comment threads with moderation options.
- Channel blocking and topic-oriented discovery search (`/discover`).

Associated files

Frontend:

- Frontend/frontend/src/components/chat/PublicChannelsPane.jsx
- Frontend/frontend/src/hooks/usePublicChannels.js

Backend:

- Backend/publicchannels/service.go
- Backend/publicchannels/handler.go

API:

- GET `/api/v1/public-channels`, `/posts`, `/discover`
- POST `/api/v1/public-channels/create`, `/update`, `/comments`, `/delete`, `/subscribe`, `/unsubscribe`, `/block`, `/unblock`
- POST `/api/v1/public-channels/posts/create`, `/reaction`, `/comment`, `/pin`, `/comments/delete`, `/comments/moderate`

Database:

- `public_channels`, `public_channel_admins`, `public_channel_subscribers`, `public_channel_posts`, `public_channel_post_reactions`, `public_channel_post_comments`, `public_channel_blocks`

Module 9: GSN - GaiaSocialNetwork (Decentralized Social Layer)

Purpose

A decentralized, federation-capable social network for microblogging, status updates and community interactions.

Capabilities

- Creation of cryptographically signed posts (`gsn_posts`) with optional image attachments.
- Node feed (local server) and following feed (subscribed contacts).
- Signed comments (`gsn_post_comments`) and emoji reactions.
- User profiles with verified status badges (operator, governance, passport).
- Client-side decryption of avatars and post images (`DecryptedAvatar.jsx` , `DecryptedGsnImage.jsx`).

Associated files

```
Frontend:
- Frontend/frontend/src/components/chat/GsnPane.jsx
- Frontend/frontend/src/components/chat/gsn/GsnPostCard.jsx
- Frontend/frontend/src/components/chat/gsn/GsnPostComposer.jsx
- Frontend/frontend/src/components/chat/gsn/GsnProfileEditor.jsx
- Frontend/frontend/src/components/chat/gsn/DecryptedAvatar.jsx
- Frontend/frontend/src/components/chat/gsn/DecryptedGsnImage.jsx
- Frontend/frontend/src/hooks/useGsn.js

Backend:
- Backend/gsn/service.go
- Backend/gsn/handler.go

API:
- POST /api/v1/gsn/posts, /posts/:id/react, /posts/:id/comment, /follow, /unfollow, /profile
- GET /api/v1/gsn/feed/node, /feed/following, /posts/:id/comments, /profile:gaia_id
- DELETE /api/v1/gsn/posts/:id, /posts/:id/comments/:commentId

Database:
- gsn_posts, gsn_post_comments, gsn_post_reactions, gsn_follows, gsn_profiles
```

Module 10: GaiaDrive & Encrypted Chunk Storage

Purpose

Combines local encrypted file storage in the browser (Origin Private File System - OPFS) with optional end-to-end-encrypted cloud synchronization through the backend.

Capabilities

- Local storage of notes and files in OPFS, encrypted with AES-GCM with keys held in the protected index.
- Cloud upload in 1 MB blocks (`file_chunks`) with SHA-256 hash validation.
- Server-side storage quotas per user (`GAIA_COM_STORAGE_USER_QUOTA_BYTES`).
- Automatic expiry cleanup of temporary uploads (14-day cloud TTL through Retention Sweeper).
- Pluggable storage backend: local file system with path jail or S3-compatible object storage.

Associated files

```
Frontend:
- Frontend/frontend/src/components/chat/DrivePane.jsx
- Frontend/frontend/src/hooks/useDrive.jsx

Backend:
- Backend/storage/storage_service.go
- Backend/storage/storage_handler.go
- Backend/storage/object_store.go
- Backend/storage/s3_object_store.go

API:
- POST /api/v1/storage/init, /chunk, /complete, /grant
- GET /api/v1/storage/download/:fileId

Database:
- file_metadata, file_access_grants, file_chunks
```

Module 11: GaiaDrop (Anonymous Text Submissions)

Purpose

Allows external or unauthenticated persons to securely submit confidential text messages to a GaiaID, providing a whistleblower/drop channel.

Capabilities

- Public submission endpoint `/api/v1/public/gaiadrop/submit`.
- Strict size limit (max. 64 KB), XSS sanitization and IP-based rate limiting.
- Recipient reads and manages submissions in protected `DropPane.jsx`.

Associated files

```
Frontend:
- Frontend/frontend/src/components/chat/DropPane.jsx
- Frontend/frontend/src/hooks/useGaiaDrop.js

Backend:
- Backend/gaiadrop/service.go
- Backend/gaiadrop/handler.go

API:
- POST /api/v1/public/gaiadrop/submit
- GET /api/v1/gaiadrop/inbox
- POST /api/v1/gaiadrop/read, /delete

Database:
- gaia_drop_submissions
```

Module 12: Federation & S2S PDU Transport

Purpose

Connects independent GaiaCom servers into a decentralized network through cryptographically signed Protocol Data Units (PDUs).

Capabilities

- Server-to-server authorization using the header:
`Authorization: X-Gaia-S2S-V1 Signature="...",KeyId="node.example.org",Timestamp="..."`
- SSRF and DNS-rebinding firewall: blocks `localhost`, RFC 1918 private ranges and loopback addresses.
- Replay protection through transaction cache (`federation_replay_guard`).
- Asynchronous federation queue (`federation_queues`) with exponential backoff and lease locking.

Associated files

```
Frontend:
- Frontend/frontend/src/components/public/NetworkHealthDashboard.jsx

Backend:
- Backend/federation/federation_service.go
- Backend/federation/federation_handler.go
```

```
- Backend/federation/federation_types.go
```

API:

```
- GET /.well-known/gaiacom/server, /nodeinfo, /nodes  
- POST /.well-known/gaiacom/s2s/v1/forward, /_gaiacom/s2s/v1/forward  
- GET /api/v1/public/nodes
```

Database:

```
- federation_servers, federation_queues, federation_replay_guard
```

Module 13: Node Registry & Core-Hash Verification

Purpose

Controlled onboarding of new server nodes, detection of software forks and preservation of version compatibility across the network.

Capabilities

- Public ping endpoint for registering new servers.
- Operator status workflow: `pending` -> `accepted`, `quarantined` or `blocked`.
- Automatic comparison of `GAIACOM_CORE_HASH` to detect outdated or modified server cores.

Associated files

Frontend:

```
- Frontend/frontend/src/components/chat/SecurityCenter.jsx (Nodesystem tab)
```

Backend:

```
- Backend/noderegistry/service.go  
- Backend/noderegistry/handler.go
```

API:

```
- POST /api/v1/public/node-registry/ping  
- GET /api/v1/public/node-registry/nodes  
- GET /api/v1/node/registry/summary  
- POST /api/v1/node/registry/secrets, /ping-main, /:domain/status
```

Database:

```
- node_registry_entries
```

Module 14: TrustMesh & Local Reputation Scoring

Purpose

Local, proof-based detection and throttling of spam and malicious behavior without violating end-to-end encryption.

Capabilities

- Creation of cryptographic abuse reports (`reports`) bound to ciphertext hashes and epoch secrets.
- Local reputation deductions (`abuse_scores`) with score-decay curves.
- Progressive friction: dynamic throughput throttling for suspicious senders up to quarantine.

Associated files

```
Frontend:
- Frontend/frontend/src/components/chat/ReaderPane.jsx (Report function)

Backend:
- Backend/trustmesh/service.go
- Backend/trustmesh/handler.go
- Core/rust/gaiacore/src/trust_mesh.rs

API:
- POST /api/v1/reports/submit

Database:
- reports, abuse_scores
```

Module 15: Governance, Gaia's Eyes & Reviewer Workflows

Purpose

Role-based management of incidents and abuse reports through reviewer panels, limited case disclosure and transparency reports.

Capabilities

- Threshold consensus for reviewers (`Senior Reviewer` , `Trusted Reviewer` , `Node Operator`).
- Bootstrap anchoring through `governance.json` and cryptographic `role_credentials` .
- Gaia's Eyes: single-use disclosure tokens that are strictly limited in time and scope for evidence in abuse cases.
- Appeals process (`abuse_appeals`) against sanctions.
- Publication of tamper-resistant transparency snapshots (`transparency_snapshots`).

Associated files

```
Frontend:
- Frontend/frontend/src/components/chat/AbuseCenter.jsx

Backend:
- Backend/governance/service.go
- Backend/governance/handler.go
- Backend/governance/bootstrap.go

API:
- GET /api/v1/governance/roles, /reports/mine, /reports/:caseID, /reviewer/cases, /node/abuse/queue, /public/transparency
- POST /api/v1/reports, /reports/:caseID/disclosures, /reports/:caseID/appeal, /gaias-eyes/grants, /reviewer/cases/:caseID/review, /node/abuse/actions, /node/transparency/snapshot, /public/gaias-eyes/consume

Database:
- governance_policies, role_credentials, role_credential_revocations, abuse_cases, abuse_case_events, abuse_disclosure_packages, abuse_disclosure_requests, gaias_eyes_grants, abuse_reviews, abuse_actions, abuse_appeals, federation_abuse_signals, transparency_snapshots
```


Module 16: GaiaShield Security Operations & Intrusion Defense

Purpose

Internal security and monitoring system for protecting node integrity, defending against attacks and providing tamper-resistant auditing.

Capabilities

- EdgeShield: pre-routing inspection against path traversal, DoS and header manipulation.
- Cryptographic audit chain (`security_audit_chain`): hash-chained, immutable audit log protected by SQLite triggers.
- Privacy-preserving redaction (`privacy.go`): removes IP addresses and sensitive payload data before logging.
- Intrusion-detection rules (`security_rules`, `security_rule_hits`) with automatic quarantine.

Associated files

```
Frontend:
- Frontend/frontend/src/components/chat/SecurityCenter.jsx

Backend:
- Backend/internal/security/shield.go
- Backend/internal/security/audit.go
- Backend/internal/security/events.go
- Backend/internal/security/privacy.go
- Backend/internal/security/security_routes.go
- Backend/internal/security/*_guard.go

API:
- GET /api/v1/public/security/health, /security/me/summary, /security/me/events, /security/me/report, /node/security/summary, /node/security/events
- POST /api/v1/security/me/events/:event_id/acknowledge

Database:
- security_events, security_event_private_context, security_rules, security_rule_hits, security_user_acknowledgements, security_quarantines, security_audit_chain, security_rate_limits
- Triggers: trg_security_events_immutable_update, trg_security_events_no_delete, trg_security_audit_chain_no_update, trg_security_audit_chain_no_delete
```

Module 17: Desktop Client Integration (Tauri 2 Bridge)

Purpose

Provides the native desktop runtime for Windows, macOS and Linux and encapsulates cryptographic operations in an isolated Rust process.

Capabilities

- No local HTTP port: communication occurs exclusively through the native Tauri IPC channel.
- HQC-256 and multi-cipher cascades execute as compiled machine code without WebAssembly overhead.
- Strict Content Security Policy and prohibition of `unsafe-eval`.

Associated files

```
Frontend:  
- Frontend/frontend/src/sovereign/nativeProvider.js
```

```
Desktop Shell:  
- DesktopClient/src-tauri/Cargo.toml  
- DesktopClient/src-tauri/tauri.conf.json  
- DesktopClient/src-tauri/src/lib.rs  
- DesktopClient/src-tauri/src/sovereign_crypto.rs
```

Module 18: Android Mobile Platform (Embedded Node)

Purpose

Enables operation as a full mobile communication node with hardware-backed security and offline capability.

Capabilities

- In-process operation of the Go backend without an open network port (`Backend/mobileapi/node.go`).
- Modular Android architecture separating presentation (Compose), kernel, security and transport.
- Hardware binding to Android Keystore and StrongBox Keymaster.
- Offline multi-hop transport over Internet, Wi-Fi Direct and Bluetooth Low Energy.

Associated files

```
Android:  
- Android/settings.gradle.kts  
- Android/platform/contracts/  
- Android/platform/kernel/  
- Android/platform/node/  
- Android/platform/security-android/  
- Android/platform/security-auth/  
- Android/app/
```

```
Backend In-Memory Bridge:  
- Backend/mobileapi/node.go  
- Backend/mobileapi/bootstrap.go  
- Backend/mobileapi/identity.go  
- Backend/mobileapi/transport.go
```

4. Dashboard & Workspace - Detailed Documentation

All Dashboard and Workspace views are centrally switched in the frontend through changes to the `currentMenu` state in `Frontend/frontend/src/App.jsx` and the `AppMainContent.jsx` renderer.

Dashboard -> Command Center (/dashboard)

- **Internal menu state:** `currentMenu === 'dashboard'`
- **Frontend file:** `Frontend/frontend/src/components/chat/DashboardPane.jsx`
- **CSS / styling:** `Frontend/frontend/src/styles/dashboard.css`, `nebula.css`, `quantum-interface.css`
- **Components used:** `LogoMark`, `VersionBadge`, metric cards, quick-access grid, PWA installation banner.
- **Responsible module:** Module 1 (Sovereign Crypto), Module 2 (Auth), Module 4 (Messaging), Module 5 (Rooms).
- **API calls:** No direct HTTP calls while rendering; uses in-memory synchronized state from `useEmails`, `useChat`, `useGaiaAuth` and `/api/v1/auth/status`.
- **Backend handlers:** `auth.GetStatus`, `operations.Liveness`.
- **Services:** `auth.AuthService`, `operations.Monitor`.
- **Database access:** Tables `users`, `identities`, `device_sessions`.
- **Authentication / authorization:** Authenticated user with a valid JWT token.
- **Data flow:**

```
User opens Dashboard
↓
App.jsx initializes DashboardPane.jsx
↓
Aggregates state (identity, unread mail, active rooms, chat metrics)
↓
Renders uplink status, PQC security level & quick actions
↓
User selects an action (e.g. open Chat) -> switches currentMenu
```

Dashboard -> Quantum Chat (/chat)

- **Internal menu state:** `currentMenu === 'chat'`
- **Frontend file:** `Frontend/frontend/src/components/chat/ChatPane.jsx`
- **CSS / styling:** `Frontend/frontend/src/styles/chat.css`, `responsive.css`, `nebula.css`
- **Components used:** `MessageActions.jsx`, emoji picker, file-upload indicator, TopSecret toggle.
- **Responsible module:** Module 4 (Quantum Chat) together with Module 1 (Sovereign Crypto).
- **API calls:**
 - Send: `POST /api/v1/messaging/send`
 - Retrieve: `GET /api/v1/messaging/inbox`
 - Read: `POST /api/v1/messaging/read`

- Reaction: `POST /api/v1/messaging/reaction`
- Delete: `POST /api/v1/messaging/delete`
- Presence: `POST /api/v1/presence/heartbeat`, `GET /api/v1/presence/status`
- **Backend handlers:** `messaging.MessagingHandler.*`, `presence.PresenceHandler.*`.
- **Services:** `messaging.MessagingService`, `presence.PresenceService`, `security.SecuritySystem`.
- **Database access:** Tables `message_envelopes`, `inboxes`, `message_proofs`, `message_reactions`, `identity_presence`.
- **Authentication:** JWT Bearer Token + verification of identity ownership (`senderIdentityId`).
- **Data flow:**

```

User sends a message in Chat
↓
ChatPane.jsx / useChat.js
↓
sovereignCrypto.js (Dual-PQC KEM + Twofish/AES-GCM or four-stage cascade)
↓
POST /api/v1/messaging/send (ciphertext Envelope only)
↓
Backend: MessagingHandler.SendMessage()
↓
MessagingService.SaveAndRouteEnvelope()
↓
SQLStore: insert into message_envelopes & inboxes
↓
JSON response (status: queued / delivered)
↓
Recipient client polls GET /api/v1/messaging/inbox
↓
Recipient worker decrypts Envelope client-side via HQC/ML-KEM
↓
Displayed in recipient's ChatPane

```

Dashboards -> GaiaMail Inbox & Folders (/inbox, etc.)

- **Internal menu state:** `currentMenu === 'inbox' | 'sent' | 'drafts' | 'starred' | 'important' | 'archive' | 'trash'`
- **Frontend files:** `Frontend/frontend/src/components/layout/ListPane.jsx` & `Frontend/frontend/src/components/chat/ReaderPane.jsx`
- **CSS / styling:** `Frontend/frontend/src/styles/listpane.css`, `reader.css`, `product-refresh.css`
- **Components used:** Mail list rows, folder navigation, star toggle, label management.
- **Responsible module:** Module 6 (GaiaMail) & Module 1 (Crypto).
- **API calls:**
 - `GET /api/v1/mailbox/messages?identityId=...&folder=...`
 - `POST /api/v1/mailbox/state`
 - `GET /api/v1/messaging/proof?messageId=...`
- **Backend handlers:** `mailbox.Handler.ListMessages`, `mailbox.Handler.UpdateStates`, `messaging.MessagingHandler.GetMessageProof`.
- **Services:** `mailbox.Service`, `messaging.MessagingService`.
- **Database access:** Tables `mailbox_states`, `message_envelopes`, `message_proofs`.
- **Authentication:** JWT Bearer Token.

- **Data flow:**

```
User selects a folder (e.g. Inbox)
↓
ListPane.jsx triggers getMailboxMessages()
↓
GET /api/v1/mailbox/messages
↓
Backend: mailbox.Handler -> mailbox.Service
↓
SQLStore: query mailbox_states JOIN message_envelopes
↓
Client receives Envelopes and decrypts subject/body in Web Worker
↓
Selecting a mail loads ReaderPane.jsx
↓
Cryptographic signature verification & GaiaProof seal display
```

Dashboard -> SMTP Mailbox (/smtp_inbox , etc.)

- **Internal menu state:** `currentMenu === 'smtp_inbox' | 'smtp_sent' | 'smtp_trash' | ...`
- **Frontend files:** `Frontend/frontend/src/components/layout/ListPane.jsx` & `Frontend/frontend/src/components/chat/ReaderPane.jsx`
- **CSS / styling:** `Frontend/frontend/src/styles/listpane.css` , `reader.css` , `gaiadrop.css`
- **Components used:** Yellow warning banners for legacy transport, sender verification.
- **Responsible module:** Module 7 (SMTP Bridge).
- **API calls:** `GET /api/v1/mailbox/messages?untrusted=true` .
- **Backend handler:** `mailbox.Handler.ListMessages` .
- **Services:** `mailbox.Service` , `smtpbridge.Service` .
- **Database access:** Tables `mailbox_states` , `message_envelopes` .
- **Authentication:** JWT Bearer Token.
- **Data flow:**

```
External mail arrives at SMTP gateway
↓
POST /api/v1/public/smtp/ingest (header checks, SPF/DKIM)
↓
SMTPBridgeService: stores Envelope with untrusted = 1
↓
Client retrieves SMTP list through ListPane.jsx
↓
ReaderPane.jsx displays a strict downgrade warning
```

Dashboard -> Group Chats & Rooms (/groups)

- **Internal menu state:** `currentMenu === 'groups'`
- **Frontend file:** `Frontend/frontend/src/components/chat/GroupChatPane.jsx`
- **CSS / styling:** `Frontend/frontend/src/styles/groupchat.css` , `chat.css`
- **Components used:** Channel list, participant bar, role manager, Slow Mode timer, pin manager.
- **Responsible module:** Module 5 (Group Chats) & Module 4 (Messaging).

- **API calls:**
 - GET `/api/v1/rooms`, `/rooms/channels`, `/rooms/pins`
 - POST `/api/v1/rooms/channels`, `/rooms/members/role`, `/rooms/pins/toggle`
 - POST `/api/v1/messaging/send` (Channel Envelope)
- **Backend handlers:** `room.Handler.*`, `messaging.MessagingHandler.SendMessage`.
- **Services:** `room.Service`, `messaging.MessagingService`.
- **Database access:** Tables `rooms`, `room_members`, `channels`, `room_pinned_messages`, `message_envelopes`.
- **Authentication:** JWT Bearer Token + room-membership and role-permission checks.
- **Data flow:**

```

User selects room and channel
↓
GroupChatPane.jsx retrieves channel messages
↓
Client-side decryption of group Envelopes
↓
Message sent via POST /api/v1/messaging/send (with channel_id)
↓
Backend checks write permission & Slow Mode
↓
Stored in message_envelopes with fanout to room inboxes

```

Dashboard -> Public Channels (`/public_channels`)

- **Internal menu state:** `currentMenu === 'public_channels'`
- **Frontend file:** `Frontend/frontend/src/components/chat/PublicChannelsPane.jsx`
- **CSS / styling:** `Frontend/frontend/src/styles/publicChannels.css`
- **Components used:** Feed cards, post editor, comment threads, discovery dialog.
- **Responsible module:** Module 8 (Public Channels).
- **API calls:**
 - GET `/api/v1/public-channels`, `/posts`, `/discover`
 - POST `/api/v1/public-channels/posts/create`, `/posts/reaction`, `/posts/comment`
- **Backend handlers:** `publicchannels.Handler.*`.
- **Services:** `publicchannels.Service`.
- **Database access:** Tables `public_channels`, `public_channel_posts`, `public_channel_post_reactions`, `public_channel_post_comments`.
- **Authentication:** JWT Bearer Token.

Dashboard -> GSN Social Network (`/gsn`)

- **Internal menu state:** `currentMenu === 'gsn'`
- **Frontend file:** `Frontend/frontend/src/components/chat/GsnPane.jsx`
- **CSS / styling:** `Frontend/frontend/src/styles/gsn.css`

- **Components used:** `GsnPostCard`, `GsnPostComposer`, `GsnProfileEditor`, `DecryptedAvatar`, `DecryptedGsnImage`.
 - **Responsible module:** Module 9 (GSN).
 - **API calls:**
 - GET `/api/v1/gsn/feed/node`, `/feed/following`, `/profile/:gaia_id`
 - POST `/api/v1/gsn/posts`, `/posts/:id/react`, `/posts/:id/comment`, `/follow`
 - **Backend handlers:** `gsn.Handler.*`.
 - **Services:** `gsn.Service`, `federation.Service`.
 - **Database access:** Tables `gsn_posts`, `gsn_post_comments`, `gsn_post_reactions`, `gsn_follows`, `gsn_profiles`.
 - **Authentication:** JWT Bearer Token + Ed25519 post signature.
-

Dashboard -> Network Health (/network_health)

- **Internal menu state:** `currentMenu === 'network_health'`
 - **Frontend file:** `Frontend/frontend/src/components/public/NetworkHealthDashboard.jsx`
 - **CSS / styling:** `Frontend/frontend/src/styles/networkHealth.css`
 - **Components used:** Status tiles, latency charts, federation table.
 - **Responsible module:** Module 12 (Federation) & Module 16 (Operations).
 - **API calls:** GET `/api/v1/public/network-health`, GET `/api/v1/public/version`.
 - **Backend handler:** `networkhealth.Handler.Dashboard`.
 - **Services:** `networkhealth.Service`.
 - **Database access:** Tables `federation_servers`, `node_registry_entries`.
 - **Authentication:** Publicly accessible (no token required).
-

Dashboard -> Security Center (/security_center)

- **Internal menu state:** `currentMenu === 'security_center'`
- **Frontend file:** `Frontend/frontend/src/components/chat/SecurityCenter.jsx`
- **CSS / styling:** `Frontend/frontend/src/styles/security.css`
- **Components used:** Audit-log table, event acknowledgment button, node-system monitor, secret generator.
- **Responsible module:** Module 16 (GaiaShield) & Module 13 (Node Registry).
- **API calls:**
 - GET `/api/v1/security/me/summary`, `/events`, `/report`
 - POST `/api/v1/security/me/events/:event_id/acknowledge`
 - Operator: GET `/api/v1/node/security/summary`, `/events`, `/node/registry/summary`
- **Backend handlers:** `security.SecurityHandler.*`, `noderegistry.Handler.*`.
- **Services:** `security.SecuritySystem`, `noderegistry.Service`.
- **Database access:** Tables `security_events`, `security_audit_chain`, `node_registry_entries`.

- **Authentication:** JWT Bearer Token; operator privileges require `governance.json` verification.
-

Dashboard -> Abuse Center (/abuse_center)

- **Internal menu state:** `currentMenu === 'abuse_center'`
 - **Frontend file:** `Frontend/frontend/src/components/chat/AbuseCenter.jsx`
 - **CSS / styling:** `Frontend/frontend/src/styles/abuseCenter.css`
 - **Components used:** Incident queue, case details, evidence decryptor, threshold voting.
 - **Responsible module:** Module 15 (Governance) & Module 14 (TrustMesh).
 - **API calls:**
 - `GET /api/v1/governance/roles, /reports/mine, /reviewer/cases, /node/abuse/queue`
 - `POST /api/v1/reports, /reviewer/cases/:caseID/review, /node/abuse/actions`
 - **Backend handlers:** `governance.Handler.*`.
 - **Services:** `governance.Service`.
 - **Database access:** Tables `abuse_cases, abuse_reviews, abuse_actions, role_credentials`.
 - **Authentication:** JWT Bearer Token + reviewer/operator role verification.
-

Dashboard -> Address Book / Contacts (/contacts)

- **Internal menu state:** `currentMenu === 'contacts'`
 - **Frontend files:** `Frontend/frontend/src/components/layout/ListPane.jsx` & `ContactProfileModal.jsx`
 - **CSS / styling:** `Frontend/frontend/src/styles/listpane.css, modals.css`
 - **Components used:** Contact cards, trust badge, key fingerprint, notes.
 - **Responsible module:** Module 3 (Identity) & Module 6 (Mailbox).
 - **API calls:** `GET /api/v1/mailbox/contacts, POST /api/v1/mailbox/contacts/save, GET /api/v1/public/identity/:gaiaID`.
 - **Backend handlers:** `mailbox.Handler.*, identity.IdentityHandler.GetPublicIdentity`.
 - **Services:** `mailbox.Service, identity.IdentityService`.
 - **Database access:** Tables `mail_contacts, identities`.
-

Dashboard -> My Profile & Security (/profile)

- **Internal menu state:** `currentMenu === 'profile'`
- **Frontend file:** `Frontend/frontend/src/components/chat/ProfilePane.jsx`
- **CSS / styling:** `Frontend/frontend/src/styles/profile.css, auth.css`
- **Components used:** Password-change form, mnemonic reveal modal, PIN/WebAuthn switches, session list.
- **Responsible module:** Module 2 (Auth) & Module 3 (Identity).
- **API calls:** `POST /api/v1/auth/change-password, /privacy, /delete-account, GET /api/v1/auth/devices`.

- **Backend handlers:** `auth.AuthHandler.*`.
 - **Services:** `auth.AuthService`.
 - **Database access:** Tables `users`, `identities`, `device_sessions`.
-

Dashboard -> GaiaDrive / Vault (`/vault`)

- **Internal menu state:** `currentMenu === 'vault'`
 - **Frontend file:** `Frontend/frontend/src/components/chat/DrivePane.jsx`
 - **CSS / styling:** `Frontend/frontend/src/styles/drive.css`
 - **Components used:** OPFS file manager, note editor, cloud-sync status, upload progress bar.
 - **Responsible module:** Module 10 (GaiaDrive).
 - **API calls:**
 - `POST /api/v1/storage/init`
 - `POST /api/v1/storage/chunk`
 - `POST /api/v1/storage/complete`
 - `GET /api/v1/storage/download/:fileId`
 - **Backend handlers:** `storage.StorageHandler.*`.
 - **Services:** `storage.Service`, `object_store.*`.
 - **Database access:** Tables `file_metadata`, `file_chunks`, `file_access_grants`.
-

Dashboard -> GaiaDrop (`/gaiadrop`)

- **Internal menu state:** `currentMenu === 'gaiadrop'`
 - **Frontend file:** `Frontend/frontend/src/components/chat/DropPane.jsx`
 - **CSS / styling:** `Frontend/frontend/src/styles/gaiadrop.css`
 - **Components used:** Submission list, integrity-hash verifier, detail viewer.
 - **Responsible module:** Module 11 (GaiaDrop).
 - **API calls:** `GET /api/v1/gaiadrop/inbox`, `POST /api/v1/gaiadrop/read`, `POST /api/v1/gaiadrop/delete`.
 - **Backend handlers:** `gaiadrop.Handler.*`.
 - **Services:** `gaiadrop.Service`.
 - **Database access:** Table `gaia_drop_submissions`.
-

Dashboard -> Mail Composer Overlay (`isComposing`)

- **Internal state:** `isComposing === true`
- **Frontend file:** `Frontend/frontend/src/components/chat/ComposerPane.jsx`
- **CSS / styling:** `Frontend/frontend/src/styles/reader.css`, `ui.css`
- **Components used:** Rich-text editor, recipient autocomplete, SMTP toggle, attachment preview.
- **Responsible module:** Module 6 (GaiaMail) & Module 7 (SMTP Bridge).

- **API calls:** `POST /api/v1/messaging/send`, `POST /api/v1/smtp/send`, `POST /api/v1/mailbox/drafts/save`.
 - **Backend handlers:** `messaging.MessagingHandler.SendMessage`, `smtpbridge.Handler.Send`, `mailbox.Handler.SaveDraft`.
-

5. CSS & UI Architecture

GaiaCom's styling architecture is based on a fully modular CSS cascade without external CSS frameworks such as Tailwind or Bootstrap. All styles are assembled in `Frontend/frontend/src/index.css` through `@import` rules in a precisely controlled order.

5.1 Global Styles & Resets

- `base.css`: CSS reset, global box-sizing rules, typography baseline (Inter / Roboto), scrollbar styling (`gaia-scrollbar`).
- `startup.css`: Preloaded styles in the HTML header; renders the fault screen (`astraea-startup-fault`) and loading states without FOUC.

5.2 Theme & Design Tokens (Dark / Light Mode)

- `theme.css`: Defines CSS custom properties on `:root`, `.dark-mode` and `.light-mode`:
 - Accent colors: `--accent-cyan: #00f2fe;`, `--accent-blue: #4facfe;`, `--accent-purple: #654ff0;`
 - Status colors: `--success: #2ed573;`, `--warning: #ffa502;`, `--danger: #ff4757;`
 - Surfaces: `--bg-primary: #0a0e17;`, `--bg-secondary: #111827;`, `--card-bg: rgba(17, 24, 39, 0.7);`
 - Blur & shadows: `--glass-blur: blur(16px);`, `--panel-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);`
- `product-refresh.css`: Refined border radii, consistent padding and sub-pixel glow effects.

5.3 Layout, Navigation & Panels

- `layout.css`: Three-column grid architecture:
 - Column 1: `.navigation-sidebar` (260px)
 - Column 2: `.mail-list-pane` (320px, collapsible)
 - Column 3: `.mail-content-pane` (flex-grow: 1)
- `ui.css`: Reusable UI primitives: buttons (`.primary-btn`, `.secondary-btn`), badges, inputs, form groups.
- `modals.css`: Modal backdrop (`.modal-backdrop`), centered containers (`.modal-container`), animations.

5.4 Module- & Page-Specific Stylesheets

- `dashboard.css`: Tile grid (`.dashboard-grid-cards`), uplink status, quick-access cards.
- `chat.css`: Chat bubbles (`.chat-bubble.sent`, `.received`), audio/file attachments, reaction picker, quote bars.
- `groupchat.css`: Group-chat-specific overrides, room header, moderation bar.
- `reader.css`: Mail reader, header metadata, attachment cards, Composer form.
- `drive.css`: GaiaDrive tree, OPFS file list, encryption indicators, cloud status.

- `profile.css` : Profile cards, mnemonic reveal grid, inactivity and privacy switches.
- `publicChannels.css` : Social broadcast feed, post cards, comment threads, discovery tiles.
- `gsn.css` : GSN Social feed, microblogging Composer, verified Passport badges, image grid.
- `auth.css` : Login, registration and unlock cards with animated background gradients.
- `onboarding.css` : Step-by-step first-user wizard.
- `security.css` : GaiaShield event logs, traffic-light status indicators, operator audit tables.
- `abuseCenter.css` : Abuse cases, evidence reviewer, threshold-voting bars.
- `networkHealth.css` : Federation status tables, ping-latency displays.
- `gaiadrop.css` : Submission forms and inbox lists for anonymous submissions.
- `vault.css` : *Legacy*: Older styling for locked note cards.

5.5 Responsive Design & Mobile Dock

- `responsive.css` : Media queries for screens below 992px and 600px:
 - Collapses side columns into off-canvas drawers (`.mobile-menu-open`).
 - Shows the bottom mobile dock (`.mobile-navigation-dock`).
 - Touch-optimized minimum heights (min. 34px / 44px) for buttons.
- `stability-refresh.css` : Defensive layout rules, scroll anchoring, prevention of horizontal overflow on mobile devices.

5.6 Visual Effects: Nebula & Quantum Interface

- `nebula.css` : Futuristic visual effects: glassmorphism, animated chrome frames (`.nebula-content-chrome`), luminous glows.
- `quantum-interface.css` : Quantum-cryptography visualizations: pulsing shields, algorithm pills (`HQC-256` , `ML-KEM-1024`).

5.7 Style Files & Component Matrix

CSS file	Affected components & views
<code>base.css</code>	Global DOM, <code>App.jsx</code> , all panes
<code>theme.css</code>	Global color scheme, Dark/Light Mode switch
<code>ui.css</code>	<code>AppFeedbackModals.jsx</code> , <code>AppModal.jsx</code> , all buttons and forms
<code>layout.css</code>	<code>NavigationSidebar.jsx</code> , <code>ListPane.jsx</code> , <code>AppMainContent.jsx</code>
<code>auth.css</code>	<code>AuthScreen.jsx</code> , <code>UnlockScreen.jsx</code> , <code>SetupWizard.jsx</code>
<code>landing.css</code>	Landing page, hero banner, welcome views
<code>listpane.css</code>	<code>ListPane.jsx</code> , folder lists, contact lists, room lists
<code>reader.css</code>	<code>ReaderPane.jsx</code> , <code>ComposerPane.jsx</code>
<code>chat.css</code>	<code>ChatPane.jsx</code> , <code>MessageActions.jsx</code>
<code>groupchat.css</code>	<code>GroupChatPane.jsx</code> , <code>GroupSettingsModal.jsx</code>
<code>drive.css</code>	<code>DrivePane.jsx</code> (GaiaDrive)

CSS file	Affected components & views
vault.css	VaultPane.jsx (Legacy)
profile.css	ProfilePane.jsx, ContactProfileModal.jsx
networkHealth.css	NetworkHealthDashboard.jsx
publicChannels.css	PublicChannelsPane.jsx, CreateChannelModal.jsx
gaiadrop.css	DropPane.jsx
modals.css	AppModalLayer.jsx, AddContactModal.jsx, CreateGroupModal.jsx
abuseCenter.css	AbuseCenter.jsx
dashboard.css	DashboardPane.jsx
security.css	SecurityCenter.jsx
gsn.css	GsnPane.jsx, all components in components/chat/gsn/
onboarding.css	FirstRunOnboarding.jsx
responsive.css	MobileNavigationDock.jsx, mobile drawers in all panes
nebula.css	Background frames and panels in AppMainContent.jsx, DashboardPane.jsx
product-refresh.css	Refined cards and tooltips throughout the workspace
quantum-interface.css	QuantumShieldModal.jsx, security badges in Chat and Dashboard
stability-refresh.css	Global scroll containers and overflow prevention

6. API Architecture & Endpoint Reference

Endpoint	Method	Frontend caller	Backend handler	Service	Data source
/livez	GET	Browser / Monitor	operations.Monitor.Liveness	operations.Monitor	In-memory status
/readyz	GET	Browser / Monitor	operations.Monitor.Readiness	operations.Monitor	DB Ping (database.db)
/metrics	GET	Monitoring agent	operations.Monitor.Metrics	operations.Monitor	In-memory counters
/api/v1/public/version	GET	VersionBadge, NetworkHealth	Anonymous route	buildinfo	buildinfo.Current()
/api/v1/public/identity/:gaiaID	GET	api.getPublicIdentity	identity.Handler.GetPublicIdentity	identity.IdentityService	identities
/api/v1/public/trust-passport/:gaiaID	GET	api.getTrustPassport	identity.Handler.GetTrustPassport	identity.IdentityService	identities, abuse_scores
/api/v1/public/nodes	GET	api.getNodes	federation.Handler.GetNodes	federation.Service	federation_servers
/api/v1/public/node-registry/ping	POST	External nodes	noderegistry.Handler.PublicPing	noderegistry.Service	node_registry_entries
/api/v1/public/node-registry/nodes	GET	NetworkHealthDashboard	noderegistry.Handler.PublicNodes	noderegistry.Service	node_registry_entries
/api/v1/public/network-health	GET	api.getNetworkHealth	networkhealth.Handler.Dashboard	networkhealth.Service	In-memory & federation_servers
/api/v1/public/gaiadrop/submit	POST	External / whistleblower	gaiadrop.Handler.Submit	gaiadrop.Service	gaia_drop_submissions
/api/v1/public/smtplib/ingest	POST	External mail server	smtplib.Handler.Ingest	smtplib.Service	message_envelopes, inboxes
/api/v1/public/transparency	GET	api.getPublicTransparency	governance.Handler.GetPublicTransparency	governance.Service	transparency_snapshots
/api/v1/public/gaias-eyes/consume	POST	api.consumeGaiasEyeKey	governance.Handler.ConsumeGaiasEyesGrant	governance.Service	gaias_eyes_grants
/api/v1/public/security/health	GET	api.getPublicSecurityHealth	security.SecurityHandler.GetPublicHealth	security.SecuritySystem	security_events
/.well-known/gaiacom/server	GET	Federation peer	federation.Handler.HandleServerDiscovery	federation.Service	Configuration
/.well-known/gaiacom/nodeinfo	GET	Federation peer	federation.Handler.HandleNodeInfo	federation.Service	Configuration & build info
/.well-known/gaiacom/nodes	GET	Federation peer	noderegistry.Handler.PublicNodes	noderegistry.Service	node_registry_entries
/.well-known/gaiacom/s2s/v1/forward	POST	Federation peer	federation.Handler.HandleS2SForward	federation.Service	message_envelopes, inboxes
/_gaiacom/s2s/v1/forward	POST	Federation peer	federation.Handler.HandleS2SForward	federation.Service	message_envelopes, inboxes
/api/v1/auth/register	POST	api.register	auth.AuthHandler.Register	auth.AuthService	users, identities
/api/v1/auth/login	POST	api.login	auth.AuthHandler.Login	auth.AuthService	users, device_sessions

Endpoint	Method	Frontend caller	Backend handler	Service	Data source
/api/v1/auth/refresh	POST	api.rotateSession	auth.AuthHandler.Refresh	auth.AuthService	device_sessions
/api/v1/auth/status	GET	api.getStatus	auth.AuthHandler.GetStatus	auth.AuthService	users, identities
/api/v1/auth/change-password	POST	api.changePassword	auth.AuthHandler.ChangePassword	auth.AuthService	users
/api/v1/auth/logout	POST	api.logout	auth.AuthHandler.Logout	auth.AuthService	device_sessions
/api/v1/auth/delete-account	POST	api.deleteAccount	auth.AuthHandler.DeleteAccount	auth.AuthService	users (CASCADE)
/api/v1/auth/privacy	POST	api.updatePrivacySettings	auth.AuthHandler.UpdatePrivacy	auth.AuthService	users
/api/v1/auth/notification-preferences	GET	api.getNotificationPreferences	auth.AuthHandler.GetNotificationPreferences	auth.AuthService	user_notification_preferences
/api/v1/auth/notification-preferences	POST	api.saveNotificationPreferences	auth.AuthHandler.SaveNotificationPreferences	auth.AuthService	user_notification_preferences
/api/v1/auth/devices	GET	api.getDeviceSessions	auth.AuthHandler.ListDevices	auth.AuthService	device_sessions
/api/v1/auth/devices/revoke	POST	api.revokeDeviceSession	auth.AuthHandler.RevokeDevice	auth.AuthService	device_sessions
/api/v1/devices/pairings	POST	api.startDevicePairing	devices.Handler.Start	devices.Service	device_pairings
/api/v1/devices/pairings/:id	GET	api.getDevicePairing	devices.Handler.Get	devices.Service	device_pairings
/api/v1/devices/pairings/:id/approve	POST	api.approveDevicePairing	devices.Handler.Approve	devices.Service	device_pairings, device_keys
/api/v1/devices/pairings/:id/consume	POST	api.consumeDevicePairing	devices.Handler.Consume	devices.Service	device_pairings
/api/v1/devices/keys	GET	api.getDeviceKeys	devices.Handler.ListKeys	devices.Service	device_keys
/api/v1/devices/recipient-keys	GET	api.getRecipientDeviceKeys	devices.Handler.RecipientKeys	devices.Service	device_keys
/api/v1/devices/keys/:id/revoke	POST	api.revokeDeviceKey	devices.Handler.RevokeKey	devices.Service	device_keys
/api/v1/identity/create	POST	api.createIdentity	identity.Handler.CreateIdentity	identity.IdentityService	identities
/api/v1/identity/me	GET	api.getMyIdentities	identity.Handler.GetMyIdentities	identity.IdentityService	identities
/api/v1/identity/human-proof	POST	api.saveIdentityHumanProof	identity.Handler.SaveHumanProof	identity.IdentityService	identities
/api/v1/messaging/send	POST	api.sendMessage	messaging.Handler.SendMessage	messaging.MessagingService	message_envelopes, inboxes
/api/v1/smtp/send	POST	api.sendSmtpMail	smtpbridge.Handler.Send	smtpbridge.Service	Outgoing SMTP / message_envelopes
/api/v1/messaging/inbox	GET	api.getInbox	messaging.Handler.GetInbox	messaging.MessagingService	inboxes, message_envelopes

Endpoint	Method	Frontend caller	Backend handler	Service	Data source
/api/v1/messaging/read	POST	api.markMessagesRead	messaging.Handler.MarkRead	messaging.MessagingService	inboxes, message_read_receipts
/api/v1/messaging/edit	POST	api.editDirectMessage	messaging.Handler.EditMessage	messaging.MessagingService	message_envelopes
/api/v1/messaging/reaction	POST	api.toggleMessageReaction	messaging.Handler.ToggleReaction	messaging.MessagingService	message_reactions
/api/v1/messaging/proof	GET	api.getMessageProof	messaging.Handler.GetMessageProof	messaging.MessagingService	message_proofs
/api/v1/messaging/delete	POST	api.deleteInboxMessage	messaging.Handler.DeleteInboxMessage	messaging.MessagingService	inboxes, message_envelopes
/api/v1/messaging/clear	POST	api.clearInboxConversation	messaging.Handler.ClearInboxConversation	messaging.MessagingService	inboxes
/api/v1/presence/heartbeat	POST	api.sendPresenceHeartbeat	presence.Handler.Heartbeat	presence.Service	identity_presence
/api/v1/presence/status	GET	api.getPresenceStatus	presence.Handler.Status	presence.Service	identity_presence
/api/v1/presence/typing	POST	api.updateTypingStatus	presence.Handler.UpdateTyping	presence.Service	In-memory typing map
/api/v1/presence/typing	GET	api.getTypingStatus	presence.Handler.TypingStatus	presence.Service	In-memory typing map
/api/v1/mailbox/messages	GET	api.getMailboxMessages	mailbox.Handler.ListMessages	mailbox.Service	mailbox_states, message_envelopes
/api/v1/mailbox/state	POST	api.updateMailboxStates	mailbox.Handler.UpdateStates	mailbox.Service	mailbox_states
/api/v1/mailbox/drafts	GET	api.getMailDrafts	mailbox.Handler.ListDrafts	mailbox.Service	mail_drafts
/api/v1/mailbox/drafts/save	POST	api.saveMailDraft	mailbox.Handler.SaveDraft	mailbox.Service	mail_drafts
/api/v1/mailbox/drafts/delete	POST	api.deleteMailDraft	mailbox.Handler.DeleteDraft	mailbox.Service	mail_drafts
/api/v1/mailbox/labels	GET	api.getMailLabels	mailbox.Handler.ListLabels	mailbox.Service	mail_labels
/api/v1/mailbox/labels/save	POST	api.saveMailLabel	mailbox.Handler.SaveLabel	mailbox.Service	mail_labels
/api/v1/mailbox/contacts	GET	api.getMailContacts	mailbox.Handler.ListContacts	mailbox.Service	mail_contacts
/api/v1/mailbox/contacts/save	POST	api.saveMailContact	mailbox.Handler.SaveContact	mailbox.Service	mail_contacts
/api/v1/mailbox/filters	GET	api.getMailFilters	mailbox.Handler.ListFilters	mailbox.Service	mail_filter_rules
/api/v1/mailbox/filters/save	POST	api.saveMailFilter	mailbox.Handler.SaveFilter	mailbox.Service	mail_filter_rules
/api/v1/mailbox/settings	GET	api.getMailSettings	mailbox.Handler.GetSettings	mailbox.Service	mail_settings
/api/v1/mailbox/settings	POST	api.saveMailSettings	mailbox.Handler.SaveSettings	mailbox.Service	mail_settings

Endpoint	Method	Frontend caller	Backend handler	Service	Data source
/api/v1/search/global	GET	api.globalSearch	mailbox.Handler.GlobalSearch	mailbox.Service	message_envelopes, mail_contacts
/api/v1/storage/init	POST	api.initUpload	storage.Handler.InitUpload	storage.Service	file_metadata
/api/v1/storage/chunk	POST	api.uploadChunk	storage.Handler.UploadChunk	storage.Service	file_chunks, ObjectStore
/api/v1/storage/complete	POST	api.completeUpload	storage.Handler.CompleteUpload	storage.Service	file_metadata
/api/v1/storage/grant	POST	api.grantFileAccess	storage.Handler.GrantAccess	storage.Service	file_access_grants
/api/v1/storage/download/:fileId	GET	api.downloadFileAttachment	storage.Handler.DownloadFile	storage.Service	file_metadata, ObjectStore
/api/v1/reports/submit	POST	api.submitReport	trustmesh.Handler.SubmitReport	trustmesh.Service	reports, abuse_scores
/api/v1/gaiadrop/inbox	GET	api.getGaiaDropInbox	gaiadrop.Handler.ListInbox	gaiadrop.Service	gaia_drop_submissions
/api/v1/gaiadrop/read	POST	api.markGaiaDropRead	gaiadrop.Handler.MarkRead	gaiadrop.Service	gaia_drop_submissions
/api/v1/gaiadrop/delete	POST	api.deleteGaiaDrop	gaiadrop.Handler.Delete	gaiadrop.Service	gaia_drop_submissions
/api/v1/gsn/posts	POST	api.createGsnPost	gsn.Handler.CreatePost	gsn.Service	gsn_posts
/api/v1/gsn/posts/:id	DELETE	api.deleteGsnPost	gsn.Handler.DeletePost	gsn.Service	gsn_posts
/api/v1/gsn/feed/node	GET	api.getGsnFeedNode	gsn.Handler.GetFeedNode	gsn.Service	gsn_posts
/api/v1/gsn/feed/following	GET	api.getGsnFeedFollowing	gsn.Handler.GetFeedFollowing	gsn.Service	gsn_posts, gsn_follows
/api/v1/gsn/posts/:id/react	POST	api.reactToGsnPost	gsn.Handler.ReactToPost	gsn.Service	gsn_post_reactions
/api/v1/gsn/posts/:id/comment	POST	api.addGsnComment	gsn.Handler.AddComment	gsn.Service	gsn_post_comments
/api/v1/gsn/posts/:id/comments	GET	api.getGsnComments	gsn.Handler.GetComments	gsn.Service	gsn_post_comments
/api/v1/gsn/posts/:id/comments/:cId	DELETE	api.deleteGsnComment	gsn.Handler.DeleteComment	gsn.Service	gsn_post_comments
/api/v1/gsn/follow	POST	api.followGsnUser	gsn.Handler.FollowUser	gsn.Service	gsn_follows
/api/v1/gsn/unfollow	POST	api.unfollowGsnUser	gsn.Handler.UnfollowUser	gsn.Service	gsn_follows
/api/v1/gsn/profile/:gaia_id	GET	api.getGsnProfile	gsn.Handler.GetProfile	gsn.Service	gsn_profiles
/api/v1/gsn/profile	POST	api.updateGsnProfile	gsn.Handler.UpdateProfile	gsn.Service	gsn_profiles
/api/v1/rooms/create	POST	api.createRoom	room.Handler.CreateRoom	room.Service	rooms, room_members, channels
/api/v1/rooms/update	POST	api.updateRoom	room.Handler.UpdateRoom	room.Service	rooms
/api/v1/rooms	GET	api.getRooms	room.Handler.GetRooms	room.Service	rooms, room_members

Endpoint	Method	Frontend caller	Backend handler	Service	Data source
/api/v1/rooms/join	POST	api.joinRoom	room.Handler.JoinRoomByHash	room.Service	room_members
/api/v1/rooms/leave	POST	api.leaveRoom	room.Handler.LeaveRoom	room.Service	room_members
/api/v1/rooms/channels	POST	api.createChannel	room.Handler.CreateChannel	room.Service	channels
/api/v1/rooms/channels	GET	api.getChannels	room.Handler.GetChannels	room.Service	channels
/api/v1/rooms/members/role	POST	api.updateMemberRole	room.Handler.UpdateMemberRole	room.Service	room_members
/api/v1/rooms/delete	POST	api.deleteRoom	room.Handler.DeleteRoom	room.Service	rooms (CASCADE)
/api/v1/rooms/search	GET	api.searchPublicRooms	room.Handler.SearchPublicRooms	room.Service	rooms
/api/v1/rooms/members/kick	POST	api.kickRoomMember	room.Handler.KickMember	room.Service	room_members
/api/v1/rooms/transfer-ownership	POST	api.transferRoomOwnership	room.Handler.TransferOwnership	room.Service	rooms, room_members
/api/v1/rooms/pins	GET	api.getRoomPinnedMessages	room.Handler.GetRoomPinnedMessages	room.Service	room_pinned_messages
/api/v1/rooms/pins/toggle	POST	api.toggleRoomMessagePin	room.Handler.ToggleRoomMessagePin	room.Service	room_pinned_messages
/api/v1/rooms/invites/create	POST	api.createRoomInviteLink	room.Handler.CreateRoomInviteLink	room.Service	room_invite_links
/api/v1/rooms/invites/join	POST	api.joinRoomViaInviteLink	room.Handler.JoinRoomViaInviteLink	room.Service	room_invite_links, room_members
/api/v1/rooms/join-requests	GET	api.getRoomJoinRequests	room.Handler.GetRoomJoinRequests	room.Service	room_join_requests
/api/v1/rooms/join-requests/create	POST	api.createRoomJoinRequest	room.Handler.CreateRoomJoinRequest	room.Service	room_join_requests
/api/v1/rooms/join-requests/moderate	POST	api.moderateRoomJoinRequest	room.Handler.ModerateRoomJoinRequest	room.Service	room_join_requests, room_members
/api/v1/rooms/moderation-logs	GET	api.getRoomModerationLogs	room.Handler.GetRoomModerationLogs	room.Service	room_moderation_logs
/api/v1/public-channels	GET	api.getPublicChannels	publicchannels.Handler.List	publicchannels.Service	public_channels
/api/v1/public-channels/create	POST	api.createPublicChannel	publicchannels.Handler.Create	publicchannels.Service	public_channels, admins
/api/v1/public-channels/update	POST	api.updatePublicChannel	publicchannels.Handler.Update	publicchannels.Service	public_channels
/api/v1/public-channels/comments	POST	api.updatePublicChannelComments	publicchannels.Handler.UpdateComments	publicchannels.Service	public_channels
/api/v1/public-channels/delete	POST	api.deletePublicChannel	publicchannels.Handler.Delete	publicchannels.Service	public_channels
/api/v1/public-channels/subscribe	POST	api.subscribePublicChannel	publicchannels.Handler.Subscribe	publicchannels.Service	public_channel_subscribers
/api/v1/public-channels/unsubscribe	POST	api.unsubscribePublicChannel	publicchannels.Handler.Unsubscribe	publicchannels.Service	public_channel_subscribers

Endpoint	Method	Frontend caller	Backend handler	Service	Data source
/api/v1/public-channels/posts	GET	api.getPublicChannelPosts	publicchannels.Handler.ListPosts	publicchannels.Service	public_channel_posts
/api/v1/public-channels/posts/create	POST	api.createPublicChannelPost	publicchannels.Handler.CreatePost	publicchannels.Service	public_channel_posts
/api/v1/public-channels/posts/reaction	POST	api.togglePublicChannelPostReaction	publicchannels.Handler.TogglePostReaction	publicchannels.Service	public_channel_post_reactions
/api/v1/public-channels/posts/comment	POST	api.createPublicChannelPostComment	publicchannels.Handler.CreatePostComment	publicchannels.Service	public_channel_post_comments
/api/v1/public-channels/posts/pin	POST	api.updatePublicChannelPostPin	publicchannels.Handler.UpdatePostPin	publicchannels.Service	public_channel_posts
/api/v1/public-channels/block	POST	api.blockPublicChannel	publicchannels.Handler.Block	publicchannels.Service	public_channel_blocks
/api/v1/public-channels/unblock	POST	api.unblockPublicChannel	publicchannels.Handler.Unblock	publicchannels.Service	public_channel_blocks
/api/v1/public-channels/discover	GET	api.discoverPublicChannels	publicchannels.Handler.Discover	publicchannels.Service	public_channels
/api/v1/public-channels/posts/comments/delete	POST	api.deleteChannelComment	publicchannels.Handler.DeleteComment	publicchannels.Service	public_channel_post_comments
/api/v1/public-channels/posts/comments/moderate	POST	api.moderateChannelComment	publicchannels.Handler.ModerateComment	publicchannels.Service	public_channel_post_comments
/api/v1/governance/roles	GET	api.getGovernanceRoles	governance.Handler.CheckRoles	governance.Service	role_credentials
/api/v1/reports	POST	api.submitAbuseReport	governance.Handler.CreateReport	governance.Service	abuse_cases, abuse_case_events
/api/v1/reports/mine	GET	api.getMyReports	governance.Handler.GetMyReports	governance.Service	abuse_cases
/api/v1/reports/:caseID	GET	api.getReportDetail	governance.Handler.GetReportDetail	governance.Service	abuse_cases, events, packages
/api/v1/reports/:caseID/disclosures	POST	api.createCaseDisclosure	governance.Handler.CreateCaseDisclosure	governance.Service	abuse_disclosure_packages
/api/v1/reviewer/cases/:caseID/disclosure-requests	POST	api.requestCaseDisclosure	governance.Handler.RequestCaseDisclosure	governance.Service	abuse_disclosure_requests
/api/v1/reports/:caseID/appeal	POST	api.submitAppeal	governance.Handler.AppelReport	governance.Service	abuse_appeals
/api/v1/gaias-eyes/grants	POST	api.createGaiasEyesGrant	governance.Handler.CreateGaiasEyesGrant	governance.Service	gaias_eyes_grants
/api/v1/reviewer/cases	GET	api.getReviewerQueue	governance.Handler.GetReviewerQueue	governance.Service	abuse_cases
/api/v1/reviewer/cases/:caseID/review	POST	api.submitReview	governance.Handler.SubmitReview	governance.Service	abuse_reviews
/api/v1/node/abuse/queue	GET	api.getNodeOperatorQueue	governance.Handler.GetNodeOperatorQueue	governance.Service	abuse_cases
/api/v1/node/abuse/actions	POST	api.applyNodeOperatorAction	governance.Handler.ApplyNodeOperatorAction	governance.Service	abuse_actions
/api/v1/node/transparency/snapshot	POST	api.createTransparencySnapshot	governance.Handler.CreateTransparencySnapshot	governance.Service	transparency_snapshots

Endpoint	Method	Frontend caller	Backend handler	Service	Data source
/api/v1/security/me/summary	GET	api.getSecuritySummary	security.SecurityHandler.GetMySummary	security.SecuritySystem	security_events
/api/v1/security/me/events	GET	api.getSecurityEvents	security.SecurityHandler.GetMyEvents	security.SecuritySystem	security_events
/api/v1/security/me/events/:id/ack	POST	api.acknowledgeSecurityEvent	security.SecurityHandler.AcknowledgeEvent	security.SecuritySystem	security_user_acknowledgements
/api/v1/security/me/report	GET	api.exportSecurityReport	security.SecurityHandler.ExportReport	security.SecuritySystem	security_events, audit_chain
/api/v1/node/security/summary	GET	api.getNodeSecuritySummary	security.SecurityHandler.GetNodeSummary	security.SecuritySystem	security_events, rules
/api/v1/node/security/events	GET	api.getNodeSecurityEvents	security.SecurityHandler.GetNodeEvents	security.SecuritySystem	security_events
/api/v1/node/registry/summary	GET	api.getNodeRegistrySummary	noderegistry.Handler.GetSummary	noderegistry.Service	node_registry_entries
/api/v1/node/registry/secrets	POST	api.generateNodeRegistrySecrets	noderegistry.Handler.GenerateSecrets	noderegistry.Service	In-memory secret generator
/api/v1/node/registry/ping-main	POST	api.pingNodeRegistryMain	noderegistry.Handler.PingMain	noderegistry.Service	Federation ping to main node
/api/v1/node/registry/:domain/status	POST	api.updateNodeRegistryStatus	noderegistry.Handler.UpdateStatus	noderegistry.Service	node_registry_entries

7. System-Wide Data Flows

Data Flow 1: Identity Creation & Dual-PQC Key Generation

```
User opens SetupWizard.jsx / AuthScreen.jsx
↓
Enters master password & desired GaiaID (@alice:node.domain)
↓
Frontend: crypto.js generates 12-word BIP-39 mnemonic
↓
BIP-39 mnemonic
├─> PBKDF2 -> X25519 identity key pair
├─> PBKDF2 -> Ed25519 signing key pair
├─> Sovereign Worker -> ML-KEM-1024 KEM key pair
└─> Sovereign Worker / Tauri Host -> HQC-256 KEM key pair
↓
Private keys are encrypted client-side with the master password (PBKDF2/AES-GCM)
and stored in local browser storage / Desktop Vault
↓
POST /api/v1/identity/create
  Payload: { gaiaId, displayName, publicRecord: { x25519, ed25519, ml_kem_1024, hqc_256 } }
↓
Backend: identity.IdentityHandler -> identity.IdentityService
↓
Integrity checks & GaiaID validation according to RFC specification
↓
Repository: stored in identities (is_active = 1)
↓
200 OK (identity registered, public keys published)
```

Data Flow 2: E2EE Quantum Chat Message Send & Receive

```
Alice types a message to Bob (@bob:remote.domain)
↓
ChatPane.jsx calls useChat.js -> sendMessage()
↓
Client retrieves Bob's public keys: GET /api/v1/public/identity/@bob:remote.domain
↓
Checked against local Key History store (Key Change Detection)
↓
Sovereign Crypto Engine (Dual-PQC KEM):
  1. X25519 Diffie-Hellman shared secret (ss_classic)
  2. ML-KEM-1024 encapsulation -> kem_ciphertext + ss_mlkem
  3. HQC-256 encapsulation -> hqc_ciphertext + ss_hqc
↓
Combiner: root_secret = HKDF-SHA3-512(ss_classic || ss_mlkem || ss_hqc, salt, transcript_aad)
↓
Cascade encryption:
  Twofish-256-EAX followed by AES-256-GCM (Accelerated)
  or four-stage cascade including Serpent & XChaCha20 (Top Secret)
↓
Envelope is signed with Alice's private Ed25519 / ML-DSA-87 key
↓
POST /api/v1/messaging/send (server sees ONLY ciphertext, KEM ciphertexts, nonces, hashes)
↓
Alice's home server: stores in message_envelopes & inboxes
↓
Check: is Bob local or on a remote node?
├─> Local: place in Bob's Inbox index
└─> Remote: federation queue creates signed PDU -> S2S POST to Bob's server
↓
Bob's client requests GET /api/v1/messaging/inbox
↓
Bob's Sovereign Worker:
  1. Decapsulates ML-KEM-1024 ciphertext with Bob's private ML-KEM key
  2. Decapsulates HQC-256 ciphertext with Bob's private HQC key
```

```

3. X25519 DH with ephemeral key
4. Root Secret derivation & cascade decryption
↓
Plaintext message displayed in Bob's ChatPane.jsx
↓
Automatic request: POST /api/v1/messaging/read (read receipt)

```

Data Flow 3: GaiaDrive Local Storage & Cloud Synchronization

```

User drops file into DrivePane.jsx
↓
useDrive.jsx intercepts the file
↓
Client-side encryption:
- Generates random 256-bit AES-GCM key & IV
- Encrypts file payload in browser storage
- Stores raw ciphertext in Origin Private File System (OPFS)
- Updates encrypted metadata index in localStorage
↓
Optional cloud synchronization (≤ 20 MB, 14-day TTL):
↓
POST /api/v1/storage/init (fileName, fileSize, mimeType, fileHash)
↓
Backend generates file_id and checks user storage quota
↓
Client splits encrypted chunks (1 MB each) and sends:
POST /api/v1/storage/chunk (Multipart FormData with chunkHash & chunkIndex)
↓
StorageService validates SHA-256 hash of the chunk
├→ Local: writes chunks to secured storage jail (uploads/<fileId>/chunk_<index>)
└→ S3: writes chunks directly to S3/MinIO bucket
↓
POST /api/v1/storage/complete
↓
StorageService assembles metadata and sets status = 'completed'

```

Data Flow 4: S2S Federation with Signed PDUs & SSRF Protection

```

Node A wants to transmit a message to Node B
↓
FederationService creates PDU:
{ pdu_id, origin: "node-a.org", destination: "node-b.org", type: "message", payload: envelope }
↓
PDU signed with Node A's Ed25519 server private key
↓
HTTP POST to Node B: /.well-known/gaiacom/s2s/v1/forward
Header: Authorization: X-Gaia-S2S-V1 Signature="...",KeyId="node-a.org",Timestamp="..."
↓
Node B EdgeShield & FederationHandler:
1. Time-window check (|timestamp - now| < 300s)
2. SSRF firewall: resolves node-a.org IP; blocks private/loopback IPs
3. Replay guard: checks federation_replay_guard; immediately rejects duplicate pdu_id
4. Signature check: verifies signature against stored Ed25519 public key for node-a.org
↓
200 OK PDU Accepted
↓
Node B places the message in the recipient's local inbox

```

Data Flow 5: Abuse Report, Gaia's Eyes Grant & Reviewer Queue

```
Recipient notices harassment or spam in ReaderPane.jsx / ChatPane.jsx
↓
Clicking "Report" opens the Abuse Center dialog
↓
Client generates GaiaProof (ciphertext hash, sender signature, server receipt timestamp)
↓
POST /api/v1/reports
  Payload: { targetGaiaID, category, severity, messageId, gaiaProof }
↓
Backend: governance.Handler -> governance.Service
↓
Creates abuse_case (status: 'pending_review')
↓
Reviewer opens AbuseCenter.jsx -> GET /api/v1/reviewer/cases
↓
Reviewer requests case evidence: POST /reviewer/cases/:id/disclosure-requests
↓
Reported user or reporter grants temporary read access via Gaia's Eyes:
POST /api/v1/gaias-eyes/grants
  Generates time-limited one-time token (TTL: 15 minutes)
↓
Reviewer consumes token via POST /api/v1/public/gaias-eyes/consume
↓
Reviewer votes: POST /reviewer/cases/:id/review (categoryVote, severityVote, recommendation)
↓
Threshold reached (e.g. 2 of 3 reviewers):
Node Operator applies action (friction throttling or temporary quarantine)
```

Data Flow 6: GaiaShield Security Event & Audit Chain

```
Unauthorized API access, BOLA attempt or rate-limit violation
↓
Backend: EdgeShieldMiddleware or SecuritySystem records violation
↓
SecuritySystem.RecordEvent():
  1. Classification: severity (INFO, WARNING, CRITICAL), category (AUTH, API, FEDERATION)
  2. Privacy redaction: IP address and User-Agent are hashed, tokens removed
  3. Audit chaining:
      event_hash = SHA256(previous_hash || event_id || timestamp || category || severity)
↓
Atomic transaction in SQLite:
  - INSERT INTO security_events
  - INSERT INTO security_audit_chain (event_id, previous_hash, event_hash)
↓
SQLite triggers enforce immutability:
  trg_security_events_immutable_update blocks modifications
  trg_security_events_no_delete blocks deletions
↓
Live warning to affected user via /api/v1/security/me/events
or to Node Operator via SecurityCenter.jsx
```

8. Shared & Core Components

The following modules and files are foundational pillars of the overall system and are reused across modules:

1. Frontend API Client (`Frontend/frontend/src/api.js`)

CORE / SHARED DEPENDENCY

- **Purpose:** Encapsulates all REST calls, manages authentication headers (`Bearer JWT`), implements automatic token rotation (`/api/v1/auth/refresh`), desktop detection (`isDesktopRuntime`) and standardized error handling.
- **Used by:** Nearly all frontend panes, hooks (`useChat` , `useEmails` , `useDrive` , `usePublicChannels` , `useGsn`) and modals.

2. Frontend Crypto Orchestrator (`Frontend/frontend/src/crypto.js` & `sovereignCrypto.js`)

CORE / SHARED DEPENDENCY

- **Purpose:** Central entry point for all cryptographic operations in the client. Encapsulates Noble curves, BIP-39 mnemonic recovery, Web Worker orchestration and Tauri IPC commands.
- **Used by:** `AuthScreen.jsx` , `ChatPane.jsx` , `DrivePane.jsx` , `ComposerPane.jsx` , `GsnPane.jsx` , `useGaiaAuth.js` .

3. Shared Rust Multi-Cipher Core (`SovereignCryptoCore/src/lib.rs`)

CORE / SHARED DEPENDENCY

- **Purpose:** Platform-independent, memory-safe implementation of the authenticated cipher cascades.
- **Used by:** `DesktopClient/src-auri` (compiled natively) and `SovereignCryptoWasm` (compiled to WebAssembly for browser workers).

4. Zero-Dependency Core Primitives (`Core/rust/gaiacore/src/lib.rs`)

CORE / SHARED DEPENDENCY

- **Purpose:** Extremely lean, standard-library-only Rust library for invariants: GaiaID validation, constant-time byte comparison, Envelope parsing and TrustMesh formulas.
- **Used by:** Rust cores and the Android kernel via JNI.

5. Backend HTTP Router & Middleware Engine (`Backend/httpx/`)

CORE / SHARED DEPENDENCY

- **Purpose:** Gin-free lightweight HTTP framework based on standard Go `net/http` . Provides CORS control, strict security headers and JSON serialization.
- **Used by:** `Backend/routes.go` , all domain handlers and operations monitoring.

6. Backend Persistence Adapter (Backend/repository/sql_store.go)

CORE / SHARED DEPENDENCY

- **Purpose:** Central wrapper around `*sql.DB`. Defines transaction boundaries, isolation levels and connects all 25 domain-specific repository parts.
- **Used by:** All backend services for relational persistence.

7. Global Domain Model (Backend/models/models.go)

CORE / SHARED DEPENDENCY

- **Purpose:** Contains all system-wide shared data structures, Envelope types, DTOs and database-row definitions.
- **Used by:** All Go packages in the backend (`auth`, `messaging`, `federation`, `room`, `mailbox`, etc.).

8. GaiaShield Security Subsystem (Backend/internal/security/shield.go)

CORE / SHARED DEPENDENCY

- **Purpose:** Central system for auditing, redaction of sensitive data, IDS rule monitoring and rate limiting.
- **Used by:** `routes.go`, `authHandler`, `messagingHandler`, `storageHandler`, `fedHandler`.

9. Secure Serialization & Type Utilities (Frontend/frontend/src/utils/safeJson.js , uuid.js , payload.js)

CORE / SHARED DEPENDENCY

- **Purpose:** Protects against prototype pollution when parsing untrusted payloads, enforces strict RFC-4122 UUID validation and sanitized data structures.
- **Used by:** Nearly all frontend components and hooks.

10. Internationalization Engine (Frontend/frontend/src/utils/i18n.jsx)

CORE / SHARED DEPENDENCY

- **Purpose:** Provides language contexts, translation functions (`t()`) and switching for 10+ languages.
- **Used by:** All user interfaces and dialogs.

9. Architecture Relationships & Mermaid Diagrams

Diagram 1: Overall System Architecture

```
graph TB
    subgraph Client_Layer ["Client Layer (End Devices)"]
        subgraph Web_Client ["Web Client (Vite / React 18)"]
            ReactUI["React UI (Panels & Modals)"]
            WebCryptoWorker["Web Worker (liboqs HQC-256)"]
            SovereignWASM["SovereignCryptoWasm (Rust Cascade)"]
            OPFS["OPFS (Local GaiaDrive Storage)"]
        end
        subgraph Desktop_Client ["Desktop Client (Tauri 2 Shell)"]
            TauriHost["Tauri 2 Rust Host"]
            NativeHQC["Native pqcrypto-hqc"]
            NativeCascade["Native SovereignCryptoCore"]
        end
        subgraph Mobile_Client ["Mobile Client (Android)"]
            ComposeUI["Jetpack Compose UI"]
            AndroidKernel["Platform Kernel"]
            AndroidKeystore["Keystore / StrongBox"]
            GoMobileNode["Embedded Go Node (Backend/mobileapi)"]
        end
    end

    subgraph Network_Edge ["Network Edge & Ingress"]
        NGINX["NGINX Reverse Proxy (TLS 1.3, CSP, Frame Deny)"]
    end

    subgraph Backend_Application ["Go Backend Application (Port :8080)"]
        HTTPX["httpx Engine (Router, CORS, Security Headers)"]
        GaiaShield["GaiaShield (Audit Chain & Privacy Redactor)"]

        subgraph Core_Services ["Core Services"]
            AuthSvc["Auth & Device Service"]
            MsgSvc["Messaging Service"]
            MailSvc["Mailbox & Scheduler Service"]
            RoomSvc["Room & Channel Service"]
            StorageSvc["Storage Service (Quotas & Chunks)"]
            FedSvc["Federation S2S Service"]
            GovSvc["Governance & Abuse Service"]
            GsnSvc["GSN Social Service"]
        end
    end

    subgraph Persistence_Layer ["Persistence & Storage Layer"]
        SQLite["SQLite Engine (WAL Mode, Max 4/32 Conns)"]
        LocalFS["Local Chunk Path (uploads/)"]
        S3Store["S3 / MinIO Object Store (Optional)"]
    end

    subgraph External_Network ["Federated Network"]
        RemoteNode["Remote GaiaCom Node"]
        SMTPGateway["Classic SMTP Email Server"]
    end

    %% Client interactions
    ReactUI --> WebCryptoWorker
    WebCryptoWorker --> SovereignWASM
    ReactUI --> OPFS
    ReactUI -->|HTTPS / API| NGINX

    TauriHost --> NativeHQC
    TauriHost --> NativeCascade
    TauriHost -->|IPC Bridge| ReactUI

    ComposeUI --> AndroidKernel
    AndroidKernel --> AndroidKeystore
    AndroidKernel --> GoMobileNode
```

```

%% Network routing
NGINX -->|Reverse Proxy :8080| HTTPX
HTTPX --> GaiaShield
GaiaShield --> Core_Services

%% Service access
AuthSvc --> SQLite
MsgSvc --> SQLite
MailSvc --> SQLite
RoomSvc --> SQLite
GovSvc --> SQLite
GsnSvc --> SQLite
StorageSvc --> SQLite
StorageSvc --> LocalFS
StorageSvc --> S3Store

%% Federation & SMTP
FedSvc <-->|Signed PDUs / S2S| RemoteNode
Core_Services -->|SMTP Ingest / Egress| SMTPGateway

```

Diagram 2: Dashboard & Navigation Architecture

```

graph LR
    User["User Interaction"] --> NavSidebar["NavigationSidebar.jsx"]

    subgraph Navigation_Sections ["Navigation Categories"]
        NavSidebar --> WorkspaceSec["Workspace"]
        NavSidebar --> NetworkSec["Network"]
        NavSidebar --> PersonalSec["Personal"]
    end

    end

    subgraph Workspace_Views ["Workspace Views"]
        WorkspaceSec --> DashboardView["DashboardPane.jsx (Command Center)"]
        WorkspaceSec --> GaiaMailView["GaiaMail (Inbox, Sent, Drafts, etc.)"]
        WorkspaceSec --> SmtMailView["SMTP Mail (Legacy Ingest / Outbox)"]
        WorkspaceSec --> ChatView["ChatPane.jsx (Quantum Chat 1-to-1)"]
        WorkspaceSec --> GroupsView["GroupChatPane.jsx (Rooms & Channels)"]
    end

    end

    subgraph Network_Views ["Network Views"]
        NetworkSec --> ChannelsView["PublicChannelsPane.jsx (Broadcasts)"]
        NetworkSec --> GsnView["GsnPane.jsx (GaiaSocialNetwork)"]
        NetworkSec --> NetHealthView["NetworkHealthDashboard.jsx"]
        NetworkSec --> SecCenterView["SecurityCenter.jsx (GaiaShield)"]
        NetworkSec --> AbuseCenterView["AbuseCenter.jsx (Abuse Center)"]
    end

    end

    subgraph Personal_Views ["Personal Views"]
        PersonalSec --> ContactsView["ListPane.jsx (Address Book)"]
        PersonalSec --> ProfileView["ProfilePane.jsx (Keys, Mnemonic, Privacy)"]
        PersonalSec --> DriveView["DrivePane.jsx (GaiaDrive / Vault)"]
        PersonalSec --> DropView["DropPane.jsx (GaiaDrop Inbox)"]
    end

    end

    subgraph Orchestration ["Renderer & Modals"]
        AppMainContent["AppMainContent.jsx (Dynamic View Switcher)"]
        AppModalLayer["AppModalLayer.jsx (Modals & Overlays)"]
    end

    end

    Workspace_Views --> AppMainContent
    Network_Views --> AppMainContent
    Personal_Views --> AppMainContent
    AppMainContent --> AppModalLayer

```

Diagram 3: Module Dependency Graph

```
graph TD
    SovereignCrypto["Module 1: Sovereign Cryptographic Engine"]
    AuthModule["Module 2: Authentication & Vault"]
    IdentityModule["Module 3: Identity & Trust Passport"]
    MessagingModule["Module 4: Quantum Chat (Direct E2EE)"]
    RoomModule["Module 5: Group Chats (Rooms & Channels)"]
    MailboxModule["Module 6: GaiaMail & Mailbox States"]
    SmtModule["Module 7: SMTP Legacy Bridge"]
    ChannelsModule["Module 8: Public Channels"]
    GsnModule["Module 9: GSN Social Network"]
    DriveModule["Module 10: GaiaDrive & Chunk Storage"]
    DropModule["Module 11: GaiaDrop Submissions"]
    FedModule["Module 12: Federation S2S Engine"]
    RegistryModule["Module 13: Node Registry"]
    TrustMeshModule["Module 14: TrustMesh Abuse Scoring"]
    GovModule["Module 15: Governance & Gaia's Eyes"]
    ShieldModule["Module 16: GaiaShield Security System"]

    %% Dependencies
    AuthModule --> SovereignCrypto
    IdentityModule --> AuthModule
    IdentityModule --> SovereignCrypto

    MessagingModule --> SovereignCrypto
    MessagingModule --> IdentityModule
    MessagingModule --> ShieldModule

    RoomModule --> MessagingModule
    RoomModule --> IdentityModule

    MailboxModule --> MessagingModule
    MailboxModule --> IdentityModule

    SmtModule --> MailboxModule
    SmtModule --> ShieldModule

    ChannelsModule --> IdentityModule
    GsnModule --> IdentityModule
    GsnModule --> SovereignCrypto

    DriveModule --> SovereignCrypto
    DriveModule --> AuthModule

    DropModule --> IdentityModule
    DropModule --> ShieldModule

    FedModule --> MessagingModule
    FedModule --> ShieldModule

    RegistryModule --> FedModule
    TrustMeshModule --> MessagingModule
    GovModule --> TrustMeshModule
    GovModule --> ShieldModule
```

Diagram 4: Backend / API / Database Pipeline

```
sequenceDiagram
    autonumber
    actor Client as Frontend Client (React/Tauri)
    participant Nginx as NGINX Reverse Proxy
    participant HTTPX as httpx Router & Middleware
    participant Shield as GaiaShield (Edge Guard)
    participant Handler as Domain Handler (e.g. Messaging)
    participant Service as Domain Service (e.g. MessagingService)
    participant Repo as SQLStore (Repository Adapter)
    participant DB as SQLite Database Engine

    Client->>Nginx: HTTPS POST /api/v1/messaging/send (Bearer Token, Ciphertext)
```

```
Nginx->>HTTPX: Proxy Pass 127.0.0.1:8080
HTTPX->>Shield: SecurityHeaders & EdgeShieldMiddleware
Shield->>Shield: Rate-limit check & Path-Jail verification
Shield->>Handler: Forward to authenticated handler
Handler->>Handler: JSON parsing & identity-rights validation
Handler->>Service: SendMessage(senderIdentityId, recipientIds, envelope)
Service->>Service: Proof generation & transcript binding
Service->>Repo: CreateEnvelope(envelope) & DeliverToInbox(inbox)
Repo->>DB: BEGIN IMMEDIATE TRANSACTION
Repo->>DB: INSERT INTO message_envelopes (...)
Repo->>DB: INSERT INTO inboxes (...)
Repo->>DB: INSERT INTO message_proofs (...)
DB->>Repo: Commit Transaction Success
Repo->>Service: Persistence confirmed
Service->>Handler: Message routed successfully
Handler->>HTTPX: httpx.WriteJSON(w, 200, {status: 'delivered'})
HTTPX->>Nginx: Response with security headers
Nginx->>Client: 200 OK
```

10. File References

For technical traceability, concrete implementation details are listed below:

- `Backend/repository/sql_store.go`: Defines the `SQLStore` data type as the concrete adapter around `*sql.DB` and implements the `repository.Store` interface from `Backend/repository/repository.go`.
 - `Backend/database/database.go`: Configures SQLite WAL mode through `PRAGMA journal_mode=WAL` and `PRAGMA foreign_keys(1)` in `sqliteDSN()`; executes `migrationStatements` to create all tables.
 - `Backend/internal/security/shield.go`: Implements `SecuritySystem`, executes `EdgeShieldMiddleware()` for all HTTP calls and controls `RunRetentionSweeper()`.
 - `Backend/internal/security/audit.go`: Uses `RecordSecurityAudit()` to calculate the cryptographic chain `event_hash = SHA256(previous_hash || event_id)` and writes to `security_audit_chain`.
 - `Backend/auth/auth_service.go`: Implements `Authenticate()`, creates password hashes using `crypto.DerivePasswordHash()` and signs JWTs via `jwt.SignToken()`.
 - `Backend/messaging/messaging_service.go`: Encapsulates `SendMessage()` and, in `generateMessageProof()`, creates the SHA-256 digest over ciphertext for storage in `message_proofs`.
 - `Backend/storage/storage_service.go`: Coordinates chunks in `SaveChunk()`, validates quotas through `CheckUserQuota()` and calls `RunFileRetentionSweeper()`.
 - `Frontend/frontend/src/sovereignCrypto.js`: Implements `sovereignSeal()` and `sovereignOpen()`, checks hex lengths through `assertHex()` and forwards calls to the Web Worker or Tauri bridge.
 - `Frontend/frontend/src/sovereign/webCrypto.worker.js`: Loads `hqc-256.runtime.js` and the WASM module `gaiacom_sovereign_wasm_bg.wasm` for isolated key computation in the browser.
 - `DesktopClient/src-tauri/src/sovereign_crypto.rs`: Exposes the five native operations `sovereign_hqc256_keypair`, `sovereign_hqc256_encapsulate`, `sovereign_hqc256_decapsulate`, `sovereign_seal` and `sovereign_open` through `#[tauri::command]`.
 - `Core/rust/gaiacore/src/lib.rs`: Validates identities in `validate_gaia_id(value: &str)` according to the pattern `@user:domain.tld` and performs timing-resistant buffer comparisons in `constant_time_eq()`.
-

11. Architecture Observations & Unused Files

The deep codebase analysis identified the following architectural peculiarities, historical artifacts and redundancies:

1. Obsolete Vault Implementation (useVault.js vs. useDrive.jsx)

- **Observation:** `VaultPane.jsx` exists in `Frontend/frontend/src/components/chat/`, and `useVault.js` exists under `hooks/`.
- **Analysis:** In `AppMainContent.jsx` and `App.jsx`, `useVault` has been fully replaced by `useDrive.jsx` and `DrivePane.jsx`. `useDrive.jsx` explicitly documents this in its header ("*Replaces useVault with GaiaDrive*"). `VaultPane.jsx` and `useVault.js` are unused historical artifacts and are no longer actively invoked by the code.

2. Duplicate Federation Forwarding Routes

- **Observation:** `Backend/routes.go` (lines 239-240) registers two equivalent forward routes:

```
router.POST("/.well-known/gaiacom/s2s/v1/forward", fedHandler.HandleS2SForward)
router.POST("/_gaiacom/s2s/v1/forward", fedHandler.HandleS2SForward)
```

- **Analysis:** `/_gaiacom/s2s/v1/forward` exists as a backward-compatibility alias for older clients, while `/.well-known/gaiacom/...` follows the standardized RFC-style discovery scheme.

3. Static System User in Database Migrations

- **Observation:** `Backend/database/database.go` (lines 494-496) creates a hard-coded user `00000000-0000-0000-0000-000000000000` (`system_remote_hub`):

```
INSERT OR IGNORE INTO users (id, username, ...) VALUES ('00000000-0000-...', 'system_remote_hub', ...)
```

- **Analysis:** This serves as a synthetic foreign-key anchor for federation objects and S2S hub messages, allowing foreign-key constraints (`ON DELETE CASCADE`) to be satisfied without a real user login.

4. Migrations: Table Alterations After Declaration in Older Blocks

- **Observation:** In `Backend/database/database.go`, several tables are first created and then extended by later migrations via `ALTER TABLE` (for example `public_channel_posts ADD COLUMN pinned_at`, `public_channels ADD COLUMN comments_enabled`).
- **Analysis:** This is the result of continuous software iteration. The migration system applies these changes incrementally; for new installations, a consolidated baseline is recommended in the long term.

5. Dual-PQC Migration Boundary (Legacy v0.1/v0.2 vs. Sovereign v1.0)

- **Observation:** `Backend/messaging/messaging_service.go` and `Frontend/frontend/src/crypto.js` contain parsing code for legacy algorithm suites (`GaiaCom/v0.1/hybrid-kem/...`).
- **Analysis:** This ensures that older message packages remain readable in the inbox during the Beta v2 phase, while new outgoing messages strictly require Sovereign v1.0 (Accelerated / Top Secret).

6. Mobile Sovereign v1.0 Message Path as a Defined Integration Boundary

- **Observation:** The native Android architecture under `Android/` has a complete module skeleton and embeds the Go node through `Backend/mobileapi` . The dual-PQC message path with native HQC-256 cascade is productively verified for browser and desktop (Tauri); mobile end-to-end integration is still in transition (Technical Beta).

7. Administrative PowerShell Scripts in the Project Root

- **Observation:** `scripts/` contains helper scripts (`accept_android_licenses.ps1` , `build_android_native.ps1`) intended for CI/CD pipelines and developers but not required at runtime for server operation.

Documentation generated and verified against the actual GaiaCom project state. Status: Sovereign Beta v2.